

PADDY LEAF DISEASE CLASSIFICATION USING A HYBRID MODEL OF RESTNET50 AND VISION TRANSFORMER

A PROJECT REPORT

Submitted by

NOORUL HASSAN M U

Under the guidance of

Dr. G. KAVITHA

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY



APRIL 2025

BONAFIDE CERTIFICATE

Certified that this project report “**PADDY LEAF DISEASE CLASSIFICATION USING A HYBRID MODEL OF RESTNET50 AND VISION TRANSFORMER**” is the bonafide work of “**NOORUL HASSAN M U (210081601045)**” who carried out the project work under my supervision. Certified further, that to the best of our knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr.G.KAVITHA

SUPERVISOR

Professor

Department of Information Technology

B.S. Abdur Rahman Crescent Institute

of Science and Technology

Vandalur, Chennai – 600 048

SIGNATURE

Dr. N. PRAKASH

HEAD OF THE DEPARTMENT

Professor

Department of Information Technology

B.S. Abdur Rahman Crescent Institute

of Science and Technology

Vandalur, Chennai – 600 048

VIVA-VOCE EXAMINATION

The viva-voce examination of the project work titled **“PADDY LEAF DISEASE CLASSIFICATION USING A HYBRID MODEL OF RESTNET50 AND VISION TRANSFORMER”**, submitted by the following students is held on

NOORUL HASSAN M U

- 210081601045

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We sincerely express our heartfelt gratitude to **Dr. T. MURUGESAN**, Vice Chancellor, B.S. Abdur Rahman Crescent Institute of Science and Technology.

We sincerely thank **Dr. N. THAJUDDIN** Pro-Vice Chancellor and **Dr. N. RAJA HUSSAIN**, Registrar, B.S. Abdur Rahman Crescent Institute of Science and Technology, for furnishing every essential facility for doing our project.

We would also like to express our sincere gratitude to **Dr. SHARMILA SANKAR**, Professor & Dean, School of Computer, Information and Mathematical Sciences, for her support.

We thank **Dr. N. PRAKASH**, Professor and Head, Department of Information Technology for his eminent support and encouragement throughout our project.

We thank our class advisor **Dr. NABEENA AMEEN**, Assistant Professor (Selection Grade), Department of Information Technology for her professional guidance and continued assistance in the process.

We thank our Supervisor **Dr. G. KAVITHA**, Professor, Department of Information Technology for her professional guidance and continued assistance in the process.

We also extend our sincere thanks to our Project Coordinator **Dr. NABEENA AMEEN** and Project Review Committee members **Dr. LATHA TAMILSELVAN**, **Dr. M. KABEER**, **Dr. P. LATCHOUMY**, **Dr. P. GNANASEKERAN**, **Dr. A. SONYA**, and **Dr. K. NALLARASU** for their continuous support to complete the project work successfully.

We would also like to thank our dear parents and our friends who directly or indirectly helped us in the completion of our project.

NOORUL HASSAN M U

ABSTRACT

The agriculture sector plays an important role in global economic stability and food security, particularly in countries such as India and China where paddy (rice) is not only a major primary food but also a major economic driving force. Crop diseases, especially in paddy fields, pose a significant risk to agricultural productivity and food security. Timely and accurate detection of these diseases is significant to prevent and mitigate crop losses and sustain rural economies. However, Traditional methods of disease detection primarily based on manual observation are labor intensive, inaccurate, and unsuitable for large-scale use. Also, traditional machine learning techniques often lack the ability to capture the complex visual patterns found in plant diseases, which makes them less effective in real-world agriculture.

To address these limitations, this project proposes a deep learning-based hybrid model that combines ResNet50 and Vision Transformer (ViT) architectures. ResNet50 model is able to learn fine-grained local spatial features using convolutional filters, and ViT model is able to learn long-range dependencies and global context relationships using self-attention mechanisms. By fusing the outputs of both networks together, the model captures a deeper and more comprehensive representation of disease features.

The system pipeline includes advanced preprocessing methods such as resizing, normalization, and contrast adjustment, followed by real-time data augmentation methods during training for improved generalization. These data augmentations such as flipping, rotation, and brightness adjustment help improve model's generalization ability by simulating diverse field conditions.

Extracted from ResNet50 and ViT features are concatenated and passed through fully connected layers, leading to a SoftMax classifier that accurately predicts the class of the disease with high accuracy. This hybrid architecture not only improves detection performance but also offers scalability and real-time applicability in precision agriculture.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	v
	LIST OF TABLES	vi
	LIST OF FIGURES	viii
	LIST OF ABBREVIATION	ix
1	INTRODUCTION	1
	1.1 GENERAL	1
	1.2 PADDY CROP AND ITS IMPORTANCE	1
	1.3 PADDY DISEASES	2
	1.4 DISEASE DETECTION	2
	1.5 WHY DEEP LEARNING FOR DISEASE DETECTION	2
	1.6 BENEFITS AND APPLICATIONS OF DEEP LEARNING	5
2	LITERATURE OVERVIEW	6
3	PROBLEM DEFINITION	13
	3.1 EXISTING SYSYTEM	13
	3.2 DRAWBACKS IN TRADITIONAL METHODS	13
	3.3 PROBLEM DEFINITION	14
	3.4 PROPOSED SYSTEM	15
	3.5 ADVANTAGES	15
4	SYSTEM DESIGN AND DEVELOPMENT	17
	4.1 ARCHITECTURE DESIGN	17

4.2	DESCRIPTION OF DATASET	18
4.3	DATA AUGMENTATION AND PREPROCESSING	20
4.4	ALGORITHM 1: PRE-AUG MODULE	20
4.5	HYBRID FUSION MODEL OVERVIEW	22
4.6	FEATURE EXTRACTION LAYER	23
4.7	CLASSIFICATION LAYER	25
4.8	ALGORITHM 2: FUSION MODEL TRAINER	27
4.9	MODEL TRAINING	29
4.10	MODEL EVALUATION	30
4.11	ALGORITHM 3: EVALUATION MODULE	31
5	IMPLEMENTATION AND RESULTS	33
5.1	EXPERIMENTAL SETUP	33
5.2	TRAINING PERFORMANCE	34
5.3	TEST SET EVALUATION	37
5.4	RESULTS SUMMARY	39
6	CONCLUSION AND FUTURE ENHANCEMENT	40
6.1	CONCLUSION	40
6.2	FUTURE ENHANCEMENT	40
	REFERENCES	41
	APPENDIX A1 – SOURCE CODE	45
	APPENDIX A2 – SCREENSHOTS	62
	APPENDIX A3 – SYSTEM REQUIREMENTS	65
	TECHNICAL BIOGRAPHY	67
	PLAGIARISM REPORT	68

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
1.1	Diagram of WHY Deep Learning	3
3.1	The Proposed Pipeline	15
4.1	Architecture Design	17
4.2	Crop Leaf Images	18
4.3	Class distribution	19
4.4	Feature Extraction Architecture	23
4.5	Classification Architecture	25
5.1	Loss over epochs	34
5.2	Accuracy over epochs	35
5.3	Learning Rate Schedule	36
5.4	Test Set evaluation	37
5.5	Confusion Matrix - Test Set	38
A2.1	Data Preprocessing Code	62
A2.2	Exploratory Data Analysis (EDA) Code	62
A2.3	Class Distribution	62
A2.4	Leaf Images	63
A2.5	Model Training Code	63
A2.6	Model Evaluation	63
A2.7	Disease Prediction Output	64
A2.8	Overall Confusion Matrix	64

LIST OF ABBREVIATIONS

CNN	Convolutional Neural Network
DL	Deep Learning
ML	Machine Learning
GPU	Graphics Processing Unit
LR	Learning Rate
ReLU	Rectified Linear Unit
SGD	Stochastic Gradient Descent
ADAM	Adaptive Moment Estimation
BCE	Binary Cross Entropy
CE	Cross Entropy
Conf. Matrix	Confusion Matrix
CR	Classification Report
TP	True Positive
FP	False Positive
ROC	Receiver Operating Characteristic
AUC	Area Under the Curve
F1- SCORE	Harmonic Mean of Precision and Recall
Acc	Accuracy

CHAPTER 1

INTRODUCTION

1.1 GENERAL

Agriculture is the backbone of human life and majority of countries economy, ensuring the continuous production of food and material to sustain day-to-day life. Agriculture significantly boosts the economies of countries and as well as gives employment to millions of workers across the globe, especially in the developing country where agriculture is the major occupation. The influence of agriculture does not only extend to the economic aspect as well as it has an impact on health, nutrition, and the environment.

But also, agriculture comes with its problems. Climate change, soil loss, infestation by pests, and the spread of plant diseases have increased the complexity of farm practice over the years. Additionally, an increasing population continues to put pressure on food production systems, and requiring more production with less resources.

To fulfill such requirements, Advances in precision agriculture, automation, and AI are changing the traditional practice of agriculture. Such things can enhance productivity, and minimize losses. Adopting such improvement is not only an optional but also a requirement to achieve sustainable agriculture.

1.2 PADDY CROP AND ITS IMPORTANCE

Paddy is a primary food crop that has a direct impact on food security in most developing countries especially in Asia. It plays an important part to ensure food security for countries like India, China, and Indonesia. Its producer supports a large agricultural workforce and rural economies. India, being a large producer, heavily depends on paddy for both domestic consumption and export. Despite its importance, paddy is vulnerable to many diseases that affect crop quality and yield. Sustaining the health and productivity of paddy crops is an important to keep national food supply chains.

1.3 PADDY DISEASES

Paddy field production is typically compromised by many crop diseases that boom in tropical and subtropical climatic conditions. While there are many diseases that infect rice crops, three major types have been commonly reported in agricultural practice and research: Bacterial Leaf Blight, Brown Spot, and Leaf Smut. In addition to healthy crop identification, these diseases represent the primary classification categories in this current study.

Bacterial Leaf Blight is a severe bacterial disease marked by water-soaked streaks that grown to dry, yellow lesions. Brown Spot is caused by a fungal disease and that appear as causing brown circular spots. Leaf Smut causes dark greenish, extended streaks along the veins of the leaves, increasingly weakening the plant's health.

These diseases not only reduce crop yield but also contribute to increased pesticide usage and economic loss. In this context, early and accurate detection plays a key role to managing their spread. Healthy rice leaves, however, possess even color and strong texture. Therefore, disease classification in paddy leaves is a key component of any smart farm system with a goal to provide sustainable and precise disease control.

1.4 DISEASE DETECTION

Plant disease detection plays a crucial role in ensuring timely intervention and minimizing crop loss. Traditional approaches depend on human expertise, which can be inconsistent, inefficient and time consuming in large scale production. Technological solutions are required to address these gaps and provide faster, and more precise diagnoses. Automation through machine learning offers promising results, particularly in image-based classification.

This leads to the growing interest in **deep learning for disease detection**.

1.5 WHY DEEP LEARNING FOR DISEASE DETECTION

Deep learning is a subset of machine learning that aims to develop algorithms based on the structure and workings of the human brain. The algorithms, also known as artificial neural networks, are capable of learning complex patterns directly from raw

data. Unlike traditional machine learning methods, which often require manually crafted features, deep learning is able to automatically extract useful features by itself as part of the training process. This ability has made it an Influenced methodology in areas such as computer vision, natural language processing, and speech recognition. Its performance in image classification applications makes it suitable for use in disease detection, where subtle differences in visual must be accurately identified.



Figure 1.1 Diagram of Why Deep Learning

In the agricultural domain, timely and accurate detection of crop diseases is important in order to ensure food security and maintaining crop yield quality. Traditional diagnostic methods, such as manual inspection or rule-based image processing, are not only time consuming but also prone to human error and inefficient at large scale. Machine learning models showed some advantages but often relied on handcrafted features and domain specific tuning limits their performance.

Deep learning, on the other hand, provides a powerful alternative by through end-to-end learning directly from image data. Since its ability to modeling complex visual patterns, it allows for better classification of plant diseases even when disease symptoms are subtle or visually similar and even in the different environmental conditions.

The use of deep learning in agriculture not only improves the detection rate but also contributes to the general objective of precision farming, also offering scalability, adaptability, and automation. It allows researchers and farmers to check the health of crops efficiently, resulting in decision making and more effective sustainable farming.

Neural Networks

The artificial neural networks form the foundation of a deep learning model with their ability to mimic the brain's structure and operations. It typically consists of an input layer, one or more hidden layers, and an output layer. Each layers hold interconnected nodes responsible for processing and passing the data. In the context of image classification, Convolutional Neural Networks (CNNs) are highly effective because of their ability to extract spatial features, while Vision Transformers (ViT) offer superior performance when it comes to learning global representations using self-attention mechanisms.

Activation Functions

Activation functions introduce non-linearity into the model so that it can learn complex patterns beyond linear relationships. ReLU (Rectified Linear Unit) is popularly used because it is very simple and effective in deep networks. In classification tasks, Softmax is used at the output layer and gives probability distributions across target classes.

Layers

Various layers are used for different specific purposes. Dense (fully connected) layers are used to learn high-level global patterns in data. Convolutional layers are essential for extracting local spatial features in images. On the other hand, transformer layers are designed to extract global dependencies by giving attention weights to various parts of an image.

Training the Model

Training a deep learning model comprises of optimizing weights using loss functions like cross-entropy, and optimizers such as Stochastic Gradient Descent (SGD) or Adam. The model learns from labeled datasets through multiple epochs. Techniques like real-time data augmentation are crucial for improving model generalization and preventing overfitting.

1.6 BENEFITS AND APPLICATIONS OF DEEP LEARNING

Deep learning is a subset of machine learning, and a game-changing technology in the field of artificial intelligence by allowing models to learn complex patterns directly from data. On other hand traditional approaches are significantly depend on handcrafted features, deep learning learns hierarchical representations of features automatically using stacks of processing layers, and that conventional machine learning cannot match.

The biggest strengths of deep learning are its potential to learn from raw data itself with minimal need of human feature engineering. This makes it particularly suitable for high dimensional data such as images and sensor inputs. Deep learning models are powerful and scalable and suitable for research purposes as well as real-world usage.

Outside of agriculture, deep learning has an impact in many sectors. In medicine, it helps to diagnosis medical image and disease analysis. In automotive sector, it powers autonomous vehicle by detecting obstacles and navigate automatically. In finance, it is used for fraud detection and stock price prediction. Also, deep learning is commonly useful in in cybersecurity, marketing automation, and virtual assistants.

CHAPTER 2

LITERATURE SURVEY

[1]In the research paper titled "Paddy Crop Disease Detection using Machine Learning" by Prajwal Gowda B.S., Nisarga M. A., Rachana M., Shashank S., and Mrs. Sahana Raj B.S., published in the International Journal of Engineering Research & Technology (IJERT) under the NCCDS 2020 Conference Proceedings, the authors propose a machine learning method to identify diseases in paddy crops using image classification. The study focuses on using basic ML algorithms in disease detection with the models being trained using features derived from paddy leaf images. The procedure involves typical steps such as data collection, preprocessing, feature extraction, and classification. However, the approach is biased more to traditional machine learning compared to deep learning, it demonstrates the effectiveness of applying computational methods in plant disease diagnosis. The paper goal to automate farming for reduce labor and supporting early disease detection to ensure crop yields.

[2]In the journal "Disease Detection in Paddy Crop using Machine Learning Techniques" by M. K. Roy, P. Manna, R. Roy, P. Roy, S. Rakshit, and M. K. H. Mondal at the 2023 International Conference on Sustainable Computing and Data Communication Systems (ICSCDS), the authors propose the application of different machine learning techniques to identify diseases in paddy crops. Transfer learning and Convolutional Neural Networks (CNNs) are the methods used in the research to achieve accurate image-based classification of paddy leaf disease. Support Vector Machines (SVMs) and other computational modeling are also applied, highlighting the use of image processing in agricultural diagnosis. Use pre-trained models and transfer to agricultural data, with the authors show good feature extraction and classification performance. The research demonstrates the power of integrating traditional ML methods with deep learning to improve prediction accuracy and makes a valuable contribution to precision agriculture.

[3]In a research paper "An Automated Convolutional Neural Network Based Approach for Paddy Leaf Disease Detection" by Md. Ashiquel Islam, Md. Nymur Rahman Shuvo, Muhammad Shamsojjaman, Shazid Hasan, Md. Shahadat Hossain, and Tania Khatun

in International Journal of Advanced Computer Science and Applications (IJACSA), the authors propose a deep learning technique by employing Convolutional Neural Networks (CNNs) for automatic detection of paddy leaf disease. The system automatically classifies healthy and diseased leaf image automatically without any need for manual feature extraction based on CNN's hierarchical feature learning capability from raw image data. Their method is based on preprocessing the image, training a CNN model, and testing against labeled data. Experiments shown effective classification accuracy using CNNs as an effective and stable tool for agricultural disease diagnosis tasks. This article highlights the importance of deep learning to the development of scalable, self-sufficient, and precise crop monitoring systems.

[4]In this paper "Detection of rice plant diseases based on deep transfer learning" by Chen J., Zhang D., Nanekaran Y.A., and Li D., in the Journal of the Science of Food and Agriculture, the authors researched for the efficienct of deep transfer learning methods to detect rice plant diseases. The research used a various of pre-trained convolutional neural networks, including AlexNet, VGGNet, and GoogLeNet, to classify various types of diseases for paddy crop. The method depend on fine-tuning the pre-trained models on a labeled dataset of paddy diseases, which reduced training time while enhancing classification performance. Transfer learning approach applied the overall visual features learned from huge collections like ImageNet to their fullest potential and mapped them to the rice disease classification domain. Deep transfer learning was superior to machine learning and shallow CNN models and has the potential to be a powerful tool in precision farming. The work demonstrates the practical applicability of deep learning based methods to mass automated agricultural disease diagnosis.

[5]In the recent work of Lamba et al. (2023), titled "A novel fine-tuned deep-learning based multi-class classifier for severity of paddy leaf diseases", published in Frontiers in Plant Science, the authors proposed a deep learning-based method to not only detect paddy leaf diseases then also classify them based on severity levels. The project used a CNN model that had been fine-tuned for multi-class classification in order to perform the task of differentiation between different levels of disease development. The data were carefully chosen and augmented for enhancing model strength, and the architecture was fine-tuned from pre-trained models to improve

performance. With disease type and severity both being part of the classification, the proposed system offers a more realistic and precise solution to actual agricultural scenarios. The model proved high accuracy in the different classes, importance the usefulness of fine-tuned deep learning techniques in plant pathology diagnosis.

[6]In the article "Grape Guard: A YOLO-based mobile application for the identification of grape leaf diseases" by S.B. Mamun et al. (2025) and published in the Journal of Electronic Science and Technology, the authors have proposed a light-weight real-time mobile app to detect grape leaf diseases with the help of the YOLO based object detection algorithm. The proposed system will help farmers and vineyard workers in detecting diseases in the field using their smartphones, taking advantage of the speed and accuracy of YOLO for bounding box-based classification and localization. The authors also optimized the YOLO model for deployment on mobile and attached it with an easy-to-use interface to enable users to take leaf photos and get real-time diagnostic responses. This work proves how object detection models such as YOLO can be applied to edge devices and real-world agricultural usage.

[7]In the paper "AgriDet: Plant Leaf Disease Severity Classification using Agriculture Detection Framework" by Arunangshu Pal and Vinay Kumar, in Engineering Applications of Artificial Intelligence (2023), the authors propose a strong framework, AgriDet, for identification and severity-level classification of plant leaf diseases. The model uses a convolutional neural network (CNN)-based architecture with fine-tuning approaches and transfer learning to improve performance on various crop types. The paper overcomes the challenge of grading severity of disease against simple presence detection, differentiating it from standard binary classifiers. The AgriDet system combines image preprocessing, deep feature extraction, and a multi-class classification pipeline that can distinguish between various phases of severity. The model was trained and tested on various datasets with high precision and recall and was put forth as a scalable, adaptable instrument for agricultural health monitoring systems.

[8]In the article "Optimized recurrent neural network-based early diagnosis of crop pest and diseases in agriculture" by V.K. Patel, Abhishek K., and S. Selvarajan, in Discover Computing (2024), the authors propose a deep learning model using Recurrent Neural

Networks (RNNs) for early detection of agricultural disease and pest infection. Unlike typical CNN-based models that are mostly depends on spatial features, the method uses RNNs to analyze sequential patterns and trend analysis of disease symptom development over time. The method uses optimization methods to achieve improved prediction accuracy as well as lower computational complexity and can support real-time crop monitoring. Integrating symptom past information with current visual data, the paper demonstrates how RNN-based sequential modeling improves early detection ability. The findings direct the model to continue precision farming by providing timely intervention and autonomous decision-support systems for pest management.

[9]In the paper titled "*Integration of Convolutional Neural Networks and Recurrent Neural Networks for Foliar Disease Classification in Apple Trees*" by Disha Garg and Mansaf Alam, published in the *International Journal of Advanced Computer Science and Applications (IJACSA)* in 2022, the authors propose a hybrid deep learning model that combines Convolutional Neural Networks (CNNs) with Recurrent Neural Networks (RNNs) for efficient classification of foliar diseases in apple trees. The CNN component is responsible for extracting local spatial features from the leaf images, capturing textures and patterns related with different disease classes, while the RNN section, specifically Long Short-Term Memory (LSTM) units, is used to process the relationships between the sequential dependencies and the extracted features. The study highlights that this hybrid model improves classification accuracy by using both spatial and sequential aspects of the data and

[10]In the paper titled "Attention-Based Recurrent Neural Network for Plant Disease Classification" by Lee SH, Goëau H, Bonnet P, and Joly A in *Frontiers in Plant Science* (2020), the authors prpose a new deep learning architecture that integrate Recurrent Neural Networks (RNNs) and an attention mechanism to classify plant diseases from leaf image. Unlike of typical CNN methods primarily depends on local spatial features, the suggested model concentrates on temporal learning by learning sequential patterns in visual information. The attention system improves the model to pay more attention to disease-related areas of the image, enhancing detection as well as classification accuracy. Experiments on publicly available datasets proved that the model outperformed benchmark methods in separating similar classes of diseases as

well as in complex backgrounds. The article represents how temporal modeling needs to be merged with visual attention in explainable and accurate plant disease diagnosis.

[11]S. Vidhanaarachchi et al. had proposed a comprehensive deep learning model for early diagnosis and severity classification of coconut leaf diseases and pest infestation in the paper "Early Diagnosis and Severity Assessment of Weligama Coconut Leaf Wilt Disease and Coconut Caterpillar Infestation Using Deep Learning-Based Image Processing Techniques" published in IEEE Access (2025). The model uses several advanced object detection models like Mask R-CNN, YOLOv5, YOLOv8, and a test YOLO11 to detect and classify disease areas with great accuracy. The system makes use of convolutional neural networks and transfer learning to improve the performance of prediction on imbalanced and small dataset. By using intensive image processing techniques and real-time camera data collection, the system is designed to offer timely management of pests and diseases. Also, early detection, the method also offers a scalable model that can be extended to other plantation monitoring systems and crops.

[12]In the research paper "A novel GCL hybrid classification model for paddy diseases" by Lamba, Baliyan, and Kukreja in the International Journal of Information Technology (2023), the authors proposed a new hybrid model called GCL (Gabor filtering + CNN + LSTM) to classify paddy crop diseases efficiently. The model starts with Gabor filtering as a image preprocessing step for leaf image texture feature extraction and augmentation, which can be appropriate in the detection of disease patterns. They are then input into a convolutional neural network (CNN) to learn the local spatial features and a long short-term memory (LSTM) network to learn sequential relations and enhance classification performance. The application of CNN and LSTM allows the system to be trained to detect sequential and local spatial patterns of disease symptoms. Their proposed model achieved high accuracy in identifying different types of diseases in paddy leaves, demonstrating that it has the potential to be a strong and effective detection system that could be used in smart farming.

[13]In the journal paper titled "A Hybrid Framework for Plant Leaf Disease Detection and Classification using Convolutional Neural Networks and Vision Transformer" in Complex & Intelligent Systems (2025), Aboelenin, Elbasheer, Eltoukhy, et al.

proposed a hybrid deep learning model integrating Convolutional Neural Networks (CNNs) and Vision Transformers (ViT) for effective plant leaf disease identification and classification. The authors used CNNs to extract local spatial details and ViTs to extract global contextual relations across the image to get rid of the disadvantages of standard CNN-only models. With the combination of these two models, the representation of features and the overall accuracy in classification were significantly improved. Based on rigorous experimentation with publicly accessible plant disease datasets, their hybrid model achieved better accuracy and reliability compared to traditional deep learning methods. The paper highlights the complementary strength of convolutional-based local feature extraction and transformer-based global attention mechanisms and demonstrates that it has a good fit for precision agriculture applications.

[14]Jayakrushna Sahoo, Sivaiah Bellamkonda, Ruchi Rani, and Sumit Kumar proposed a novel method for the detection of corn disease in their 2025 MethodsX published work using attention-augmented few-shot learning and a VGG16 convolutional neural network. In an effort to reverse the prevalent challenge of scarce annotated agricultural datasets, the authors came up with a model that simulates human insight in line with the need for having few examples on which to train in order to generalize effectively. They utilized a pre-trained VGG16 model on a corn disease dataset and added an attention mechanism to prioritize features effectively. Prototypical few-shot learning was also used to help the model learn novel classes from limited samples. The system could achieve an excellent classification rate of 98.25%, further testifying to its efficacy. The research highlighted the practical advantage of this method toward early disease detection, especially in situations where the lack of available data prevents the use of classical deep learning systems. This hybrid architecture not only minimizes the requirement for enormous amounts of labeled data but also provides a high-performance, scalable solution for real-world farm settings.

The entire literature survey on plant disease detection represents growing use of machine learning (ML) and deep learning (DL) models to solve the challenges of agricultural diagnosis. A few studies used traditional ML algorithms like SVM, KNN,

and Random Forest, normally using hand-crafted features, but their performance was typically limited by the difficulty and inconsistency of data. On the other hand, the latest studies focus on deep learning models such as CNNs, RNNs, YOLO variants, and hybrid models, which support better spatial and related feature extraction. Transfer learning using pretrained models (e.g., VGG16, ResNet, and MobileNet) and ensemble learning techniques more improved detection accuracy and generalization. Initial breakthroughs brought forth attention mechanisms, few-shot learning, and transformer models (such as ViT) to counter the drawbacks of limited datasets and to achieve long-range dependencies. Deployments covered a variety of crops such as paddy, corn, apple, and coconut, for disease classification, and severity prediction. Works also offered the real-time and mobile deployment systems, capability of field-based deployment. The combination of next-generation preprocessing, data augmentation, and real-time training augmentation methods significantly improved model robustness. In summary, literature shows a clear standard moves toward traditional methods to deep learning based hybrid systems and offers reason to use the method of integration between CNNs and transformers to provide accurate, scalable, and robust detection of crop disease—justification of the motive and architecture of this work.

CHAPTER 3

PROBLEM DEFINITION

3.1 EXISTING SYSTEM

In agricultural domains, early detection and classification of plant diseases are critical to reducing yield losses. Traditionally, in crop disease detection systems, farmers and agricultural experts heavily depend on manual visual inspection to identify disease in crops. This is all subjective and open to human errors through visual checks and manual examination. Farmers might rely on noticeable symptoms such as color change, bendy, and spot changes. While some diseases are quite easy to determine, others are to identify. These observations are depended on experience, so these are subjective, inconsistent, and often fail under large-scale or early-stage disease conditions.

From the technology point of view, existing models typically have machine learning methods like SVM, Naive Bayes, and decision trees. Even though the models help in streamlining the disease detection procedure, they are depending on manual feature extraction and do not perform well in detecting diseases accurately from raw image data. Deep learning (DL) took over the scene and enabled the development of more sophisticated models like Convolutional Neural Networks (CNNs) to handle more complex image datasets.

3.2 DRAWBACKS IN TRADITIONAL METHODS

Inefficient and Error-Prone Manual Methods

Manual inspection of crop diseases involves physical inspection of leaves and crop patterns by human farmers or experts. The process is extremely time-consuming, prone to human error, and not consistent across regions and crops. Weather, lighting, and disease progress further affect decision, leading to misdiagnosis or delayed treatment.

Slow and Inefficient

Even though ML and DL methods have brought the task of classifying disease features automated in several aspects, the majority of current models either computationally

expensive, non-real-time, or fail to generalize for all crops and diseases. Traditional feature extractions are requiring manual feature extraction in traditional ML models. Even the precise deep learning approaches are depended upon large labeled data and prolonged training times. Deployment on a real-time basis, especially for use on mobile or edge hardware, is not as of yet within reach.

Limited Scalability and High Costs

Most DL models are learned and tested in controlled laboratory environments with clean, organized dataset, and so difficult to scale over various environmental conditions, leaf morphology, and image qualities found in the field. In addition, training such models from scratch is computationally expensive, and most pre-trained models experience domain mismatch. Few well performed in learning and domain adaptation techniques attempt to solve these problems but are experimental and do not have strong levels in open field deployments.

3.3 PROBLEM DEFINITION

Early and accurate diagnosis of crop diseases in agriculture domain is an important role to ensure the crop health, prevention of yield loss, and food security, especially in the case of staple crops like paddy. While recent advances in machine learning and deep learning have been encouraging, most existing models are not able to generalize across different crop species, environments, and disease types. In addition, such models require large labeled datasets and are computationally expensive, which makes them less scalable and practical in the real-time agricultural situation. The challenge is to develop a consistent, accurate, and efficient model that can diagnose and classify multiple crop diseases with minimal data need. A strategy that combines the strengths of both CNNs and Transformer-based models, along with transfer learning and best augmentation practices, is a possible solution to overcome this constraint.

3.4 PROPOSED SYSTEM

The proposed system introduces a deep learning-based hybrid fusion model with the strong local feature extraction ability of ResNet50 (Convolutional Neural Network) and global feature extraction ability of Vision Transformer (ViT) for effective paddy leaf disease classification. The system begins with input image preprocessing and data augmentation to improve the robustness of the model and prevent overfitting. After this step, the preprocessed images are passed to the fusion model, where both models extract features of image data. ResNet50 extract features of local feature patterns and ViT extract features of global pattern. Then features are concatenated and passed to a fully connected classifier for prediction. Training is optimized using an OneCycle Learning Rate scheduler to enable faster convergence and generalization. Finally, the trained model is tested against standard metrics and live data to ensure its effectiveness. This architecture is specifically designed to be scalable, accurate, and more dynamic in live agricultural environments where traditional single-model solutions fall short.

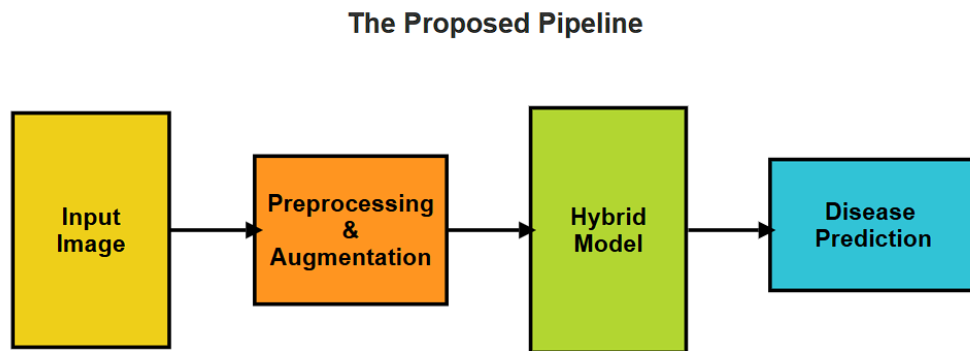


Figure 3.1 The Proposed Pipeline

3.5 ADVANTAGES

Enhanced Accuracy through Feature Fusion

By combining CNN and Transformer models, the system is able to use both local and global spatial patterns of the image, resulting in a richer and more discriminative feature representation that significantly enhance classification accuracy.

Robustness with Limited Data

The system is designed to combine data augmentation and transfer learning, along with a fine-tuned ResNet50 backbone, enabling it to generalize well even with relatively smaller datasets. This makes it suitable for agricultural domains where labeled dataset is often typically small and expensive to label.

Scalability and Real-World Deployment

The design of the fusion model allows it possible to implement it on any platform—edge devices, cloud-based, or mobile. Additionally, its performance under different field conditions and varying lighting or leaf quality also makes it feasible for real-world farming scenarios.

CHAPTER 4

SYSTEM DESIGN AND DEVELOPMENT

4.1 ARCHITECTURE DESIGN

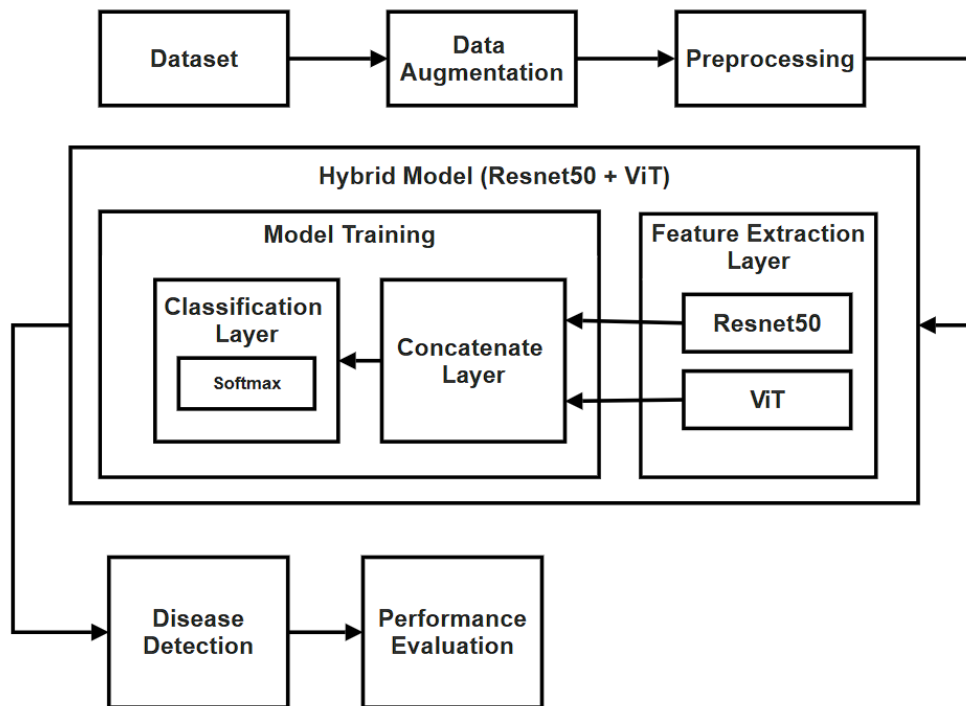


Figure 4.1: Architecture diagram

The figure 4.1 is the architecture of the proposed model. It begins with the process of collecting a dataset where the raw images are acquired. The images are then data augmented for increasing the robustness and diversity of the collected dataset and, then, passing through the steps of preprocessing which includes normalization, contrast enhancement and resizing of input. High resolution images are used as an input to a hybrid model which combines ResNet50 and Vision Transformer (ViT) for strong feature extraction. The output from both the models is combined in a concatenation layer to acquire a strong feature vector. The combined representation is passed through the classification layer, where there is a sequence of dense layers and activation functions leading to the final prediction of disease. The predicted outcomes are then examined with performance metrics to evaluate the strength of the model.

4.2 DESCRIPTION OF DATASET

The data used in this study was sourced from Kaggle and then cleaned and modified and small changes for improved training stability and readability. It contains high-resolution leaf images of paddy crops, which are grouped into four corresponding classes relevant to paddy disease detection. The paddy crop images are in the figure 4.2, and the class distribution in the training set is visually represented in bar chart in figure 4.3 below with comparatively equal samples across all classes, which helps to reduce class imbalance issues during training.

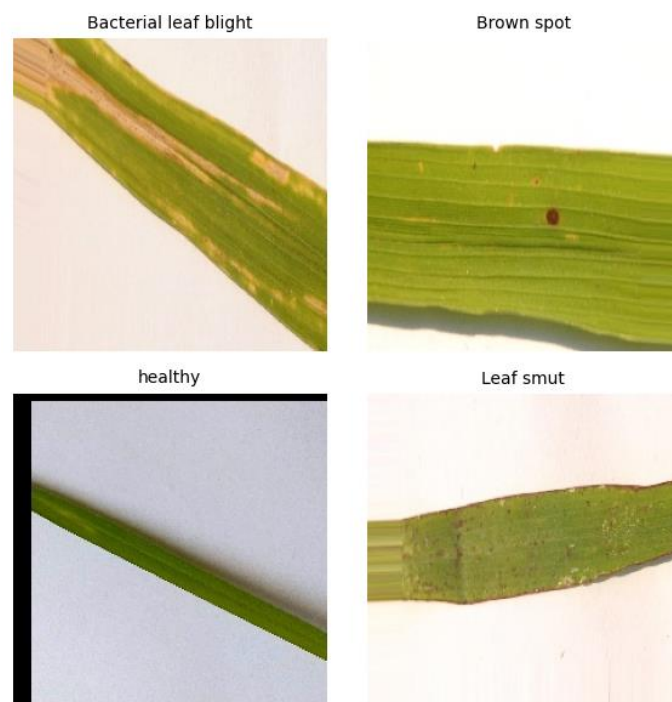


Figure 4.2: Crop Leaf Images

- **Bacterial Leaf Blight:** Water-soaked spots on leaves, which later enlarge and turn into yellow in leaf tips and edges.
- **Brown Spot:** Unusual in the shape of brown or circular brown spots with typically caused by fungal infections.

- **Healthy:** Describes leaves that are free from disease, infestation, or stress and serve as reference class.
- **Leaf Smut:** Takes the form of small, dark greenish, narrow linear spots or lines on the leaves, usually caused by fungal infection in moist conditions.

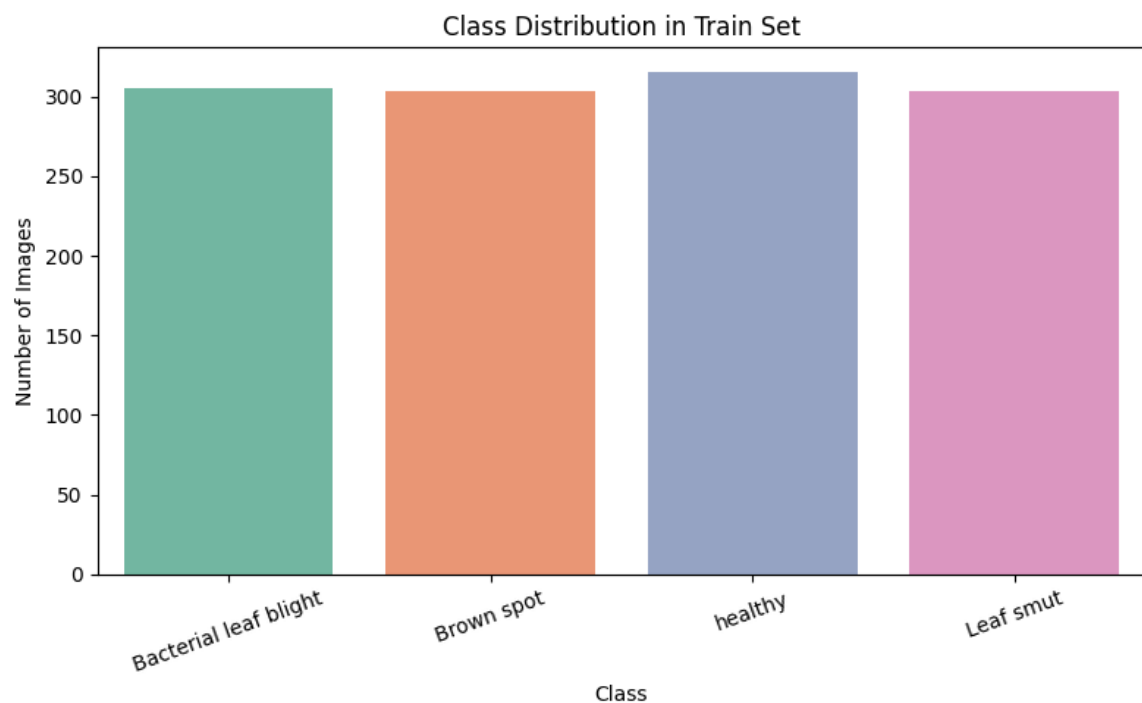


Figure 4.3: Class distribution

The dataset offers a solid base for the training and testing of deep learning models for plant disease classification by containing a vast variety of disease symptoms and healthy ones.

4.3 DATA AUGMENTATION AND PREPROCESSING

To improve the generalization ability of the model and prevent overfitting, data augmentation techniques are applied to the training images. These augmentations simulate real-world image variations and introduce variety to the dataset without requiring additional labeled data. The applied transformations are random rotations of ± 10 degrees, horizontal and vertical shifts by up to 10%, horizontal mirroring with a 50% probability, and brightness and contrast changes between 0.9 and 1.1. But surprisingly, zoom, shear, and vertical mirroring are not involved in order to keep the physical patterns of leaves, which are important for disease classification. These techniques help the model learn invariant features and gain robustness against image biases.

Preprocessing is an important process that normalizes input images before feeding into the model. It involves resizing each image to a fixed size of 224x224 pixels, changing their color space to RGB, and transforming them into tensors for model consumption. An extra optional normalization step is also performed to normalize pixel values to the range 0-1 to enhance quicker convergence during training. All of preprocessing and augmentation pipeline is encapsulated in Algorithm 1 – Data Preprocessing and Augmentation, which ensures uniform processing of training, validation, and test data. While augmentations are only done on the training set, preprocessing is done uniformly on all data subsets, thereby maintaining evaluation integrity and having a consistent input format.

4.4 ALGORITHM 1 – PRE-AUG MODULE

Algorithm 1: Data Preprocessing and Augmentation is designed to simplify both data preprocessing and augmentation of image data. It resizes and normalizes images into a standard format ready for model training. Rotation, flipping, and brightness/contrast manipulation are applied to generate variation in the training set. This enables improved generalization and forms effective DataLoaders for any split of the dataset.

Pseudo Code:

Class DataPreprocessor:

Function init(image_size=(224, 224), normalize=True):

- Set target image size

- Enable pixel normalization

Function preprocess_images()::

- For each image in the dataset folder:

 - Load image in RGB color mode

 - Resize image to target size

 - Convert image to tensor format

 - If normalize is True:

 - Scale pixel values to range [0, 1] by dividing by 255

- Return preprocessed image dataset

Function augment_images(image_batch):

- For each image in image_batch:

 - Randomly rotate within ± 10 degrees

 - Randomly shift horizontally and vertically ($\pm 10\%$)

 - Randomly flip image horizontally with 50% probability

Randomly adjust brightness (range 0.9 to 1.1)

Randomly adjust contrast (range 0.9 to 1.1)

Avoid zoom, shear, and vertical flipping

Return augmented image batch

Function `get_data_loaders(batch_size=32)`:

Apply preprocessing to train, validation, and test directories

Apply augmentations only to training set

Create and return `DataLoaders` for train, validation, and test sets

4.5 HYBRID FUSION MODEL OVERVIEW

The hybrid model proposed integrates the efficiency of both Convolutional Neural Networks (CNNs) and Vision Transformers (ViTs) to deliver strong and accurate crop disease detection. ResNet50, for example, a deep Convolutional Neural Network, is used to extract local and spatial characteristics effectively, whereas Vision Transformer (ViT) is specifically effective at capturing long-distance dependencies and global context signs in the image. By combining the two architectures, the system can leverage both local detailed feature extraction and overall image understanding.

The integration of ResNet50 and ViT is motivated by their complementary advantages. CNNs like ResNet are expert models of visual recognition tasks with strong inductive biases like locality and translation invariance. On the other hand, ViTs, though less biased, learn more global-dependent representations from data. The hybrid model concatenates the feature vectors of the two branches before inputting them into a dense classification network. This fusion representation enhances the ability of the model to distinguish between visually similar leaf diseases, leading to improved performance and reliability in real-world agricultural applications.

4.6 FEATURE EXTRACTION LAYER

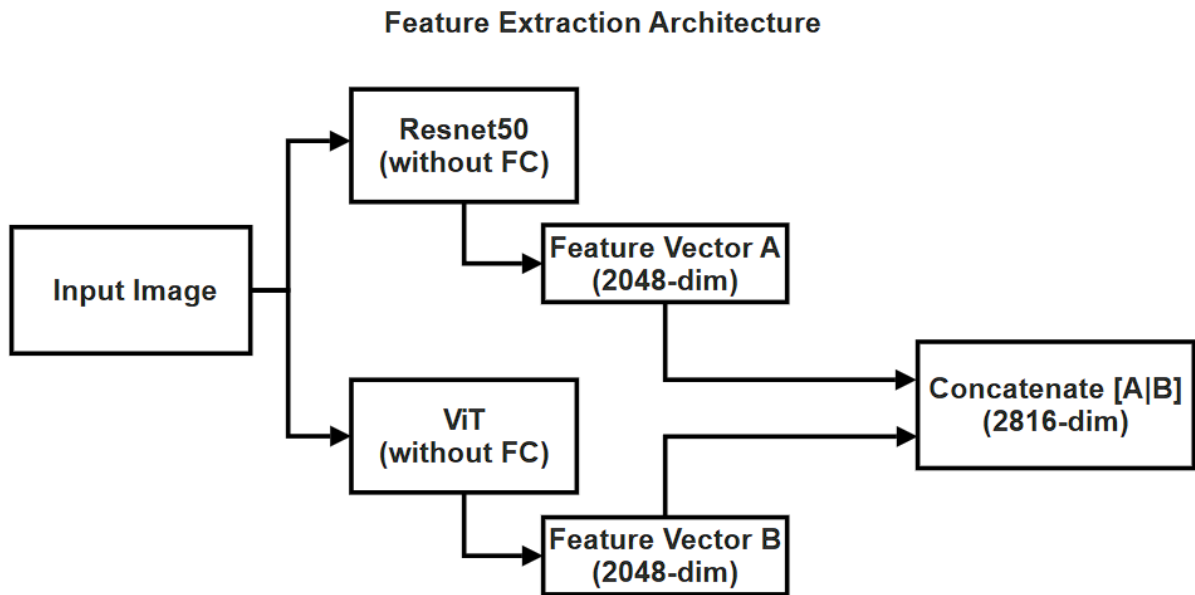


Figure 4.4: Feature Extraction Architecture

Parallel processing of input images with ResNet50 and Vision Transformer (ViT) without their final classification layers is the feature extraction process. The two models separately extract deep features of leaf image patterns (Feature Vector A and Feature Vector B), and these are combined to form a common feature space. High-dimensional, augmented representation is then passed into the classification layer. The figure 4.4 illustrates this structure, highlighting the way that the two models collaboratively operate towards diverse and informative feature extraction

ResNet50

ResNet50 is a 50-layer convolutional neural network that is well-known for its application of residual connections which address the problem of the vanishing gradient during the training of a deep network. It contains several blocks of convolution in which every block contains identity shortcut connections to enable the flow of gradients easily. In the hybrid model, ResNet50 is employed without its fully connected (FC) layers as a feature extractor alone. It is used for the extraction of local spatial features such as textures, edges, and patterns that have an important role in the detection of disease specific leaf symptom characters such as spots, blights, or smuts.

Vision Transformer (ViT)

Vision Transformer (ViT) brings a transformer model to vision tasks over the segmentation of an input image into fixed-size patches like tokens in words and encoding them in a very similar way as word tokens in NLP. The patches are linearly embedded and are pass into one or several layers of transformer encoders composed of multi-head self-attention and feedforward blocks. Unlike CNNs, ViT captures global relations between distant parts of an image. In the hybrid configuration, ViT supports ResNet50 by viewing the leaf globally and extracting structural patterns or subtle variances that are not locally focused. This offers a strong symptom interpretation of disease across the image.

Feature Vectors A and B

After processing the input image ResNet50 outputs Feature Vector A, and ViT outputs Feature Vector B, with a high-dimensional representation (2048-dim and 2048-dim, respectively). These vectors have the unique characteristics learned by each model. Feature Vector A holds dense local spatial patterns and information, while Feature Vector B has contextual as well as positional relations across the image Together, they provide a balanced and comprehensive feature set for the classification task.

Concatenation Layer

The Concatenation Layer combines Feature Vector A and Feature Vector B into a single, merged vector (2816-dim after dimension reduction). This fusion layer acts as the bridge between feature extraction and classification to allow the model to use the strength of both architectures at the same time. With the combination of CNN-extracted local spatial features and transformer extracted global feature representations, the concatenated feature vector provides a more discriminative and solid input to the classifier. This improves the model's generalization ability and accuracy, especially for visually similar or complex disease categories.

4.7 CLASSIFICATION LAYER

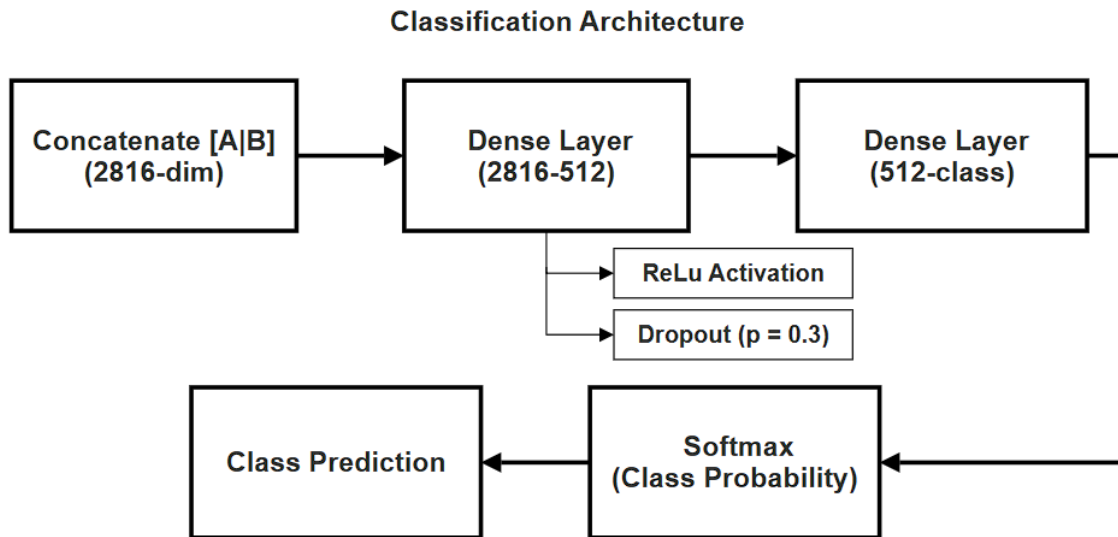


Figure 4.5: Classification Architecture

The Classification Layer is the final level of the hybrid fusion framework that converts extracted features into accurate class predictions. Subsequent the extraction of the deep features of ResNet50 and Vision Transformer (ViT) and concatenated into one vector with 2816 dimensions, then the concatenated representation is input into the classification head. The work of this layer is to convert the high-dimensional, dense features to one of the target classes: Brown spot, Bacterial leaf blight, Healthy, or Leaf smut.

The pipeline starts with a Dense layer that transforms the input of 2816 dimensions to 512 by dimensionality reduction. Here is actual feature selection step through which the network can focus on the most informative and relevant patterns. The first dense layer is followed by a ReLU activation function to introduce non-linearity so the model can more effectively learn complex patterns and interaction between features.

In order to avoid overfitting, especially since the model is parameter rich due to both ResNet and ViT branches, a Dropout layer with a probability of 0.3 is applied. During training time, this will randomly dropout 30% of neurons in the layer, forcing the network to learn more strong representations.

Following the dropout, the well-tuned features are passed into another Dense layer that outputs a vector of dimensions equal to the classes (4 in our example). This vector contains raw class logits (scores), which are then passed through a Softmax activation function. Softmax converts logits into probabilities that sum up to 1 (0.01, 0.03, 0.02, 0.94), with each containing the model's confidence for the corresponding class.

This entire chain of operations – Dense → ReLU → Dropout → Dense → Softmax – allows the model to process from abstract, high-dimensional features to good class predictions. It is efficient, and directly related to the hybrid fusion methodology on the outcome, that both convolutional features and attention-based features play an important part in taking the final decision.

Concatenated Feature Vector (2816-dim)

This feature vector is result from output of concatenating the output of ResNet50 (2048 features) and ViT (768 features). Both parts of this feature vector are picking up different features:

- ResNet50 picks up local spatial and texture features, such as edges, shapes, and local patterns.
- ViT picks up global contextual dependencies within the image via attention mechanisms.

By fusing both of them together, the model benefits from local detail and global information at the same time and so it makes the feature representation very expressive.

Dense Layer (2816 → 512)

It is a fully connected layer and acts as an abstraction and compression unit. It learns to focus on the combination most relevant of features by compressing dimension. It basically removes noises and redundant information in the input vector and preserves what's most important to classification.

ReLU Activation

Rectified Linear Unit is one of the popular activation functions introducing non-linearity in the network. It helps the model learn complex patterns by only activating neurons for useful features. ReLU also avoids vanishing gradients during training.

Dropout Layer ($p = 0.3$)

A regulation method that permanently disables a subset of the neurons while training. This prevents the model from overfitting on some lines in the network and improves its ability to generalize to new data.

Dense Layer ($512 \rightarrow \text{num_classes}$)

This is the final linear transformation from the flattened feature vector to the class score. There is one unit per class in this layer that produces a raw logit (score) before activation.

Softmax Activation

The softmax function transforms the logits into probabilities, and hence the output can be read as class predictions. It chooses the class with the highest probability as the final prediction for an image.

4.8 ALGORITHM 2 – FUSION MODEL TRAINER

This algorithm states a whole pipeline to train and build a proposed hybrid model that combines feature extraction and classification. It starts with the initialization of critical hyperparameters like batch size, learning rate, and epochs. The data are loaded and preprocessed by resizing, tensor conversion, and augmentation when necessary. The feature extraction is handled by two parallel branches of ResNet50 and Vision Transformer (ViT) individually, which learn contextual and spatial features. These are concatenated end to end to create an extended 2816-dimensional representation, which is pass into the fully connected classification head that is includes of dense layers, ReLU activation, dropout regularization, and end softmax output for multi-class prediction. The model is trained using Adam optimizer, CrossEntropy loss, and OneCycleLR learning rate scheduler. During training, the model is subjected to

iterative optimization across epochs, with validation used to monitor the performance and avoid overfitting. The most recent trained model is evaluated against a test set to calculate accuracy, loss, and class-wise performance in terms of metrics such as confusion matrix and classification report.

Pseudo Code:

Class FusionModel:

Function init(batch_size, num_epochs, learning_rate):

- Set batch_size, num_epochs, learning_rate

- Set image_size = (224, 224)

- Set device = GPU if available else CPU

Function load_datasets(train_path, val_path, test_path):

- Apply resizing and tensor conversion

- Load datasets from folders (train, val, test)

- Create DataLoaders for each set

Function build_model(num_classes):

- Load ResNet50 without classifier

- Load pretrained Vision Transformer without head

- Concatenate ResNet and ViT features

- Add fully connected classifier:

 - Input = 2048 + 768 → Dense(512) → ReLU → Dropout → Dense(num_classes)

Function load_pretrained_resnet(weights_path):

- Load saved weights into ResNet branch of the fusion model

Function compile_model():

- Define loss function = CrossEntropy

- Define optimizer = Adam

- Define scheduler = OneCycleLR (auto-adjust LR during training)

Function train():

```
For each epoch from 1 to num_epochs:
    Set model to training mode
    For each batch in training data:
        Forward pass → Compute loss
        Backward pass → Update weights
        Step the scheduler
    Set model to evaluation mode
    For each batch in validation data:
        Forward pass → Compute loss
    Store and log training and validation accuracy/loss
```

```
Function evaluate_test_set():
    Set model to evaluation mode
    Predict on test set
    Compute classification report and confusion matrix
```

```
Function save_model(path):
    Save the trained fusion model to disk
```

```
Function plot_results():
    Plot training vs. validation loss
    Plot training vs. validation accuracy
    Plot learning rate schedule
```

4.9 MODEL TRAINING

Training is significant in the hybrid fusion model to learn relevant features for successful classification. The model was trained for a specified number of epochs, it enabling to progressively improve its internal weights through forward and backward propagation. An epoch consists of a single pass through the training set where the model makes predictions, calculates loss, and updates its parameters through backpropagation. In this project, a batch sized (usually 32 images) was applied for balancing training stability and memory consumption.

The learning rate (LR) was dynamically updated by the OneCycleLR scheduler, which fine-tunes the LR over training. The LR starts with a low value, increases to a highest point, and then linearly declines until training ends. This periodic learning rate policy avoids local minima and speeds up convergence, leading to better performance and generalization. As well, the Adam optimizer was used to possess an adaptive learning algorithm, which updates the learning rate for each parameter automatically based on historical gradients, hence being applicable for deep architectures such as hybrid models.

Dropout regularization was used to prevent overfitting and increase robustness in the classification layers at a learning rate of 0.3. This was used to randomly disable a subset of the neurons during training, leaving the model with no choice but to learn redundant paths and therefore improve generalization over unseen examples. ReLU activation was used after each dense layer to add non-linearity to the network and allow complex patterns learned in the concatenated feature space.

Above all, live augmentation was used in training to improve the model's generalization ability. Random augmentations like horizontal flip, few-degree rotations, brightness and contrast adjustments, and random translations were used on each training batch. These image augmentations are real-world differences and render the model to handle such noise, so it turns out to perform uniformly for all various input samples.

All training activity was performed on a GPU to speed up process. The model was also tested on the validation set at every epoch in an effort to track performance and to keep tabs on metrics like accuracy and loss. On this response, the model adjusted its learning through the optimizer and scheduler, continually improving its ability at classification.

4.10 MODEL EVALUATION

After the successful training, the hybrid ResNet50 and ViT model was extensively tested on the training, validation, and test sets to evaluate its performance and generalization. evaluation was conducted using the standard classification metrics such as accuracy, precision, recall, and F1-score, which were computed using

classification_report of sklearn.metrics. The ability of the model to differentiate among various classes was also demonstrated using the confusion matrix, giving a comprehensive overview of true versus predicted labels.

The trained model was loaded with PyTorch's load_state_dict() function for weights and architecture consistency. No gradient computation (torch.no_grad()) was employed in the evaluation loop, allowing for computational efficiency and preventing weight updates unnecessarily. For each sample, the model performed a forward pass, producing class probabilities, from which the most likely class was determined using torch.max.

All the data sets (train, validate, and test) were pass through the model with individual DataLoaders that made ensuring batch-wise processing. The predicted labels and true labels were added and passed to classification_report() and confusion_matrix() to obtain final evaluation metrics. A confusion matrix plotted using ConfusionMatrixDisplay, which provided a visual representation of performance by class and helped in identifying any consistent misclassifications.

In summary, this evaluation confirmed that the model not only learned from training data but also maintained good performance on validation and test sets, confirming the effectiveness of the fusion approach. This end-to-end test pipeline confirms confidence in deploying the model in the real world with an emphasis on making correct and explainable predictions.

4.11 ALGORITHM 3: EVALUATION MODULE

This algorithm defines the evaluation procedure by a trained hybrid model. The model is placed in evaluation mode and predictions are made batch-wise on the test set. Ground truths and predictions are utilized to calculate a classification report and confusion matrix. Results are shown with proper labels and color mapping. A summary that gives overall accuracy and useful insights into class-wise performance and misclassifications finishes off the process.

Pseudo Code:

Class ModelInference:

Function init(model, test_loader, class_names, device):

- Load trained model
- Set model to evaluation mode
- Set test_loader and class_names

Function predict():

- Initialize empty list for predictions and true labels
- For each batch in test_loader:
 - Move images to device
 - Perform forward pass
 - Get predicted class using argmax
 - Store predictions and true labels
- Return all predictions and true labels

Function evaluate(predictions, labels):

- Generate classification report (Precision, Recall, F1-score per class)
- Generate confusion matrix

Function display_results():

- Show classification report on console
- Plot confusion matrix with class labels
- Set title and color map
- Rotate x-axis labels for clarity

Function summarize():

- Output overall accuracy and insights on misclassifications
- Optionally highlight class-wise performance issues

CHAPTER 5

IMPLEMENTATION AND RESULT

5.1 EXPERIMENTAL SETUP

In this work, we have designed an end-to-end pipeline for paddy leaf disease classification based on data preprocessing, model development, training, and validation both on local and cloud platforms. The initial preprocessing and augmentation of the dataset were done locally using Albumentations, where all images were resized to 224×224 pixels and normalized. To enhance the strength of the model and avoid overfitting, a series of augmentations such as rotation, flip, brightness, and contrast transformations were applied directly to the training set. The preprocessed data were saved separately for train, val, and test splits to ensure consistency during training.

Model training and testing were performed using Google Colab since it offers access to GPU acceleration (NVIDIA T4). A novel hybrid deep learning model was proposed by fusing ResNet50 and Vision Transformer (ViT-B/16). The pre-trained ResNet50 branch, with fine-tuned weights, was employed as a feature extractor by removing its classification head. Similarly, the ViT model head was removed for straightforward feature concatenation. The 2816-dimensional fused feature vector was input into a fully connected classifier head designed for multi-class classification. The model was trained using Adam optimizer with learning rate 1e-3 and OneCycleLR scheduler for 25 epochs with the batch size 32.

During training, accuracy and loss were tracked for training and validation sets, with visual plots generated to monitor learning dynamics. Once completion of training, the model was saved and later loaded for final evaluation. Dataloaders for each dataset split were created using PyTorch's ImageFolder with only resizing and tensor conversion performed during runtime. This streamlined setup ensured efficient use of GPUs as well as stable results throughout all phases of the pipeline.

5.2 TRAINING PERFORMANCE

In Epoch-wise Analysis The model was trained for 25 epochs. These metrics were plotted to visualize the learning dynamics of the model.

1. Training and Validation Loss
2. Training and Validation Accuracy
3. Learning Rate schedule

Training and Validation Loss:

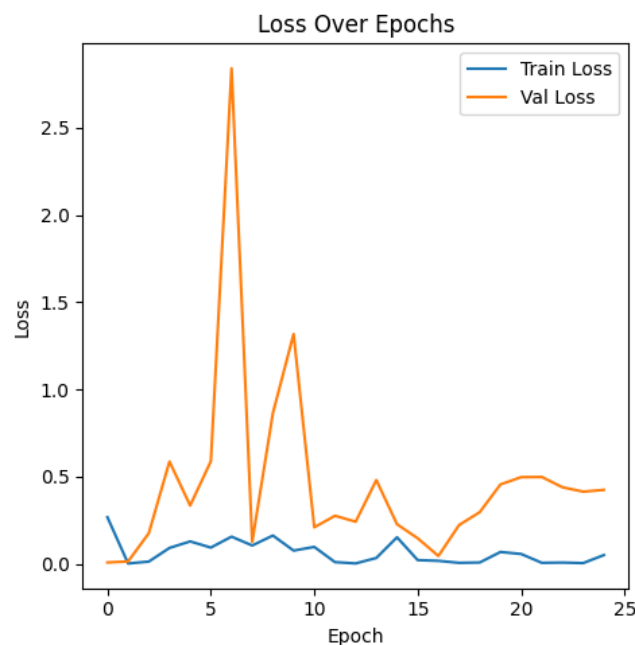


Figure 5.1: Loss over epochs

Training loss kept decreasing from 0.2678 in Epoch 1 to below 0.01 in the later epochs, indicating effective learning. Validation loss was extremely low (0.0087) at the beginning, indicating good generalization, but rose exponentially from Epochs 4 to 7 to an all-time high of 2.8401 likely due to overlearning or unstable learning. Validation loss stayed constant at approximately 0.4–0.5 from Epoch 8 onwards, indicating improved performance. This recovery indicates that the model learned and bypassed early training instability. Overall, the patterns of loss suggest a good-optimized training process.

Training and Validation Accuracy:

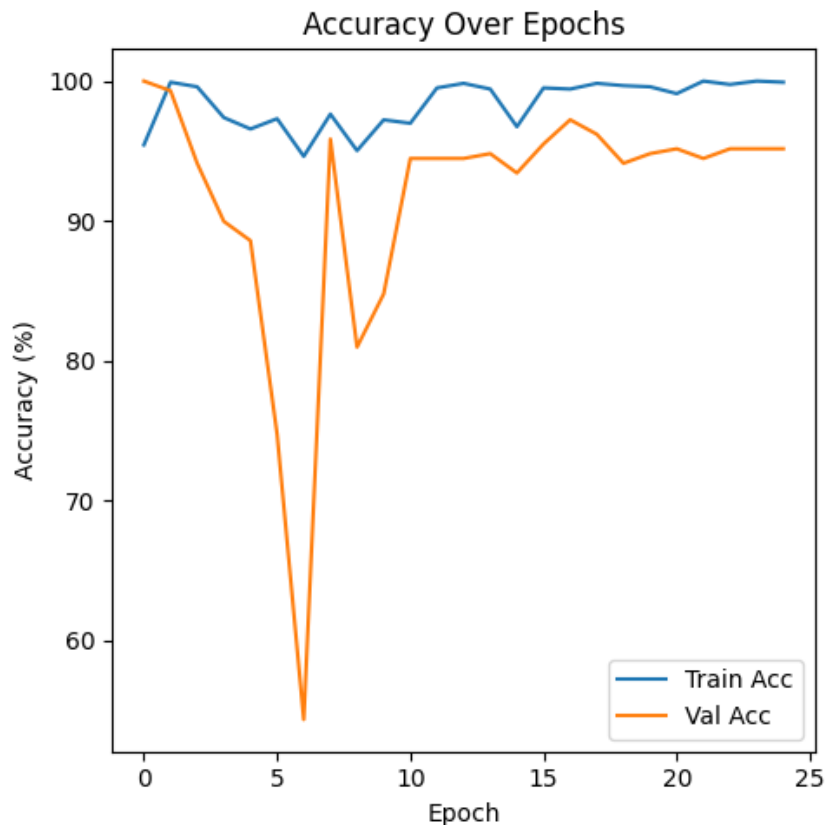


Figure 5.2: Accuracy Over Epochs

During the beginning of training (Epoch 1), the model showed strong early generalization with training loss 0.2678 and very low validation loss of 0.0087. Afterwards, during Epochs 4-7, the validation loss shot up quickly to 2.8401 in Epoch 7 with signs of potential overfitting or learning instability. The peak was concurrent with the peak of the learning rate. After Epoch 8, validation loss gradually bettered and normalized to 0.4–0.5. Conversely, training loss continually went down and dropped below 0.01 for the latter epochs. Such behavior shows how the model tolerates early fluctuation and ensures a stable process of training. The model generalizes well throughout as a whole despite early instability.

Learning Rate schedule:

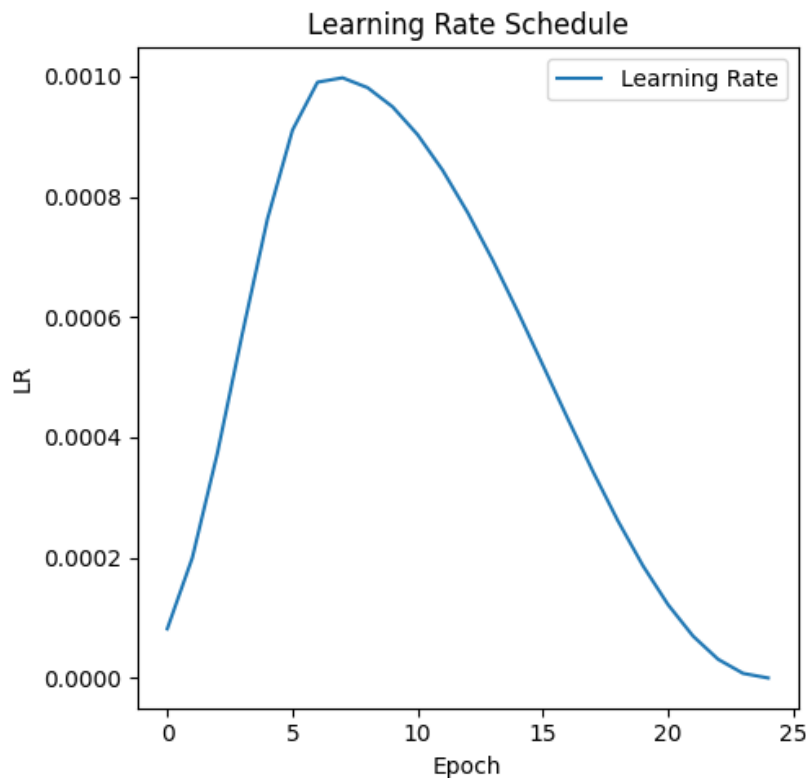


Figure 5.3: Learning Rate Schedule

The learning rate was in a cyclical increase, first 0.000082 in Epoch 1 and increasing gradually to around 0.001 at Epoch 7, then again decreasing to zero. The peak in the learning rate matched with a steep increase in the validation loss at Epoch 7 to a high of 2.8401, indicating that the model had overestimated for a short time the optimal weights due to the high learning rate. After this plateau, the validation loss resumed its rise and stabilized, showing that the model had stabilized and readapted its weights for the subsequent epochs. The cyclical learning rate schedule allowed the model to explore the parameter space initially, and the decelerating reduction aided convergence and improved generalization by the subsequent epochs. The learning rate behavior was most vital in achieving a balance between exploration and convergence during training.

5.3 TEST SET EVALUATION

 Test Set Accuracy: 98.15%				
 Classification Report:				
	precision	recall	f1-score	support
Bacterial leaf blight	1.00	1.00	1.00	55
Brown spot	1.00	0.98	0.99	54
Leaf smut	0.98	0.94	0.96	54
healthy	0.95	1.00	0.97	53
accuracy			0.98	216
macro avg	0.98	0.98	0.98	216
weighted avg	0.98	0.98	0.98	216

Figure 5.4: Test set evaluation

Test Accuracy:

The model's test accuracy was a high 98.15%, indicating outstanding generalization performance on novel data. This metric guarantees the model's success in correctly classifying novel cases and its robustness beyond the training and validation phases.

Classification Report:

The classification report includes precision, recall, F1-score, and support for each class. It provides further insight into model performance by individual categories:

- Bacterial leaf blight scored perfect scores (1.00 across all the metrics), showcasing the strength of the model in identifying this disease.
- Brown spot was detected with almost perfect accuracy (1.00) and a slightly lower recall (0.98), which led to an F1-score of 0.99.
- Leaf smut saw a slight drop in recall (0.94), coupled with an F1-score of 0.96, which shows periodic misclassification with other classes.
- Healthy leaves were well remembered (1.00) but a bit less accurate (0.95), with an F1-score of 0.97.

All the measurement are around at 0.98, which reflects consistent performance across majority and minority classes.

Confusion Matrix:

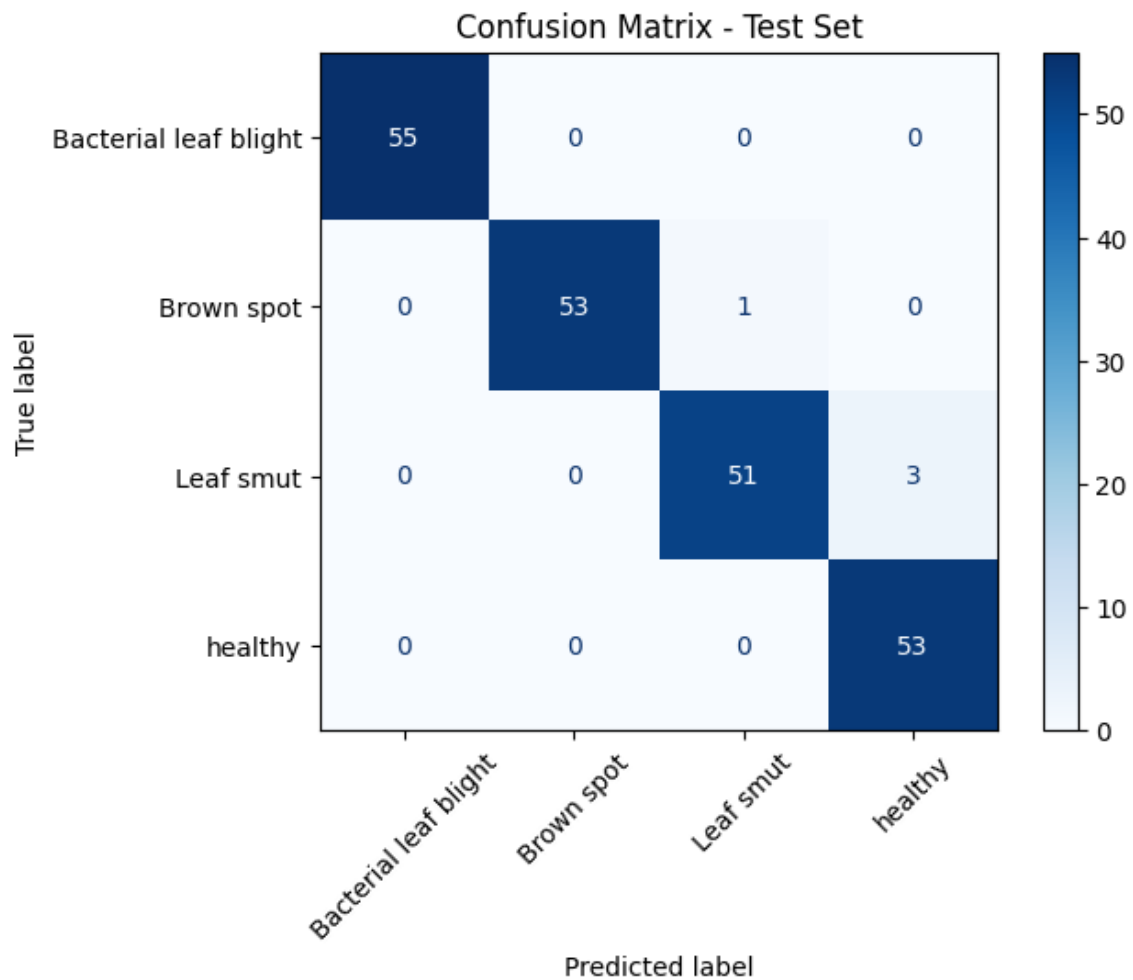


Figure 5.4: Confusion Matrix - Test Set

Confusion matrix shows great performance for all classes with the model correctly identifying nearly all the test samples. Bacterial leaf blight and healthy classes were correctly predicted with no misclassifications whatsoever. Brown spot had a single sample misclassified as leaf smut, while leaf smut had three samples misclassified as healthy. This indicates slight confusion between healthy and leaf smut classes. The general matrix structure is extremely diagonal, representing good predictions. The model is excellently discriminative, especially in distinguishing disease types. These results are in line with the high-test accuracy and balanced performance seen within the classification report.

5.4 RESULTS SUMMARY

The trained model was also well-performing on the test set with an overall high accuracy of 98.15%, indicating good generalization capacity. The classification report shows uniformly high precision, recall, and F1-scores across all classes with a perfect one for Bacterial leaf blight and healthy leaves. Leaf smut was with some misclassifications, yet still a high F1-score of 0.96. This confusion matrix confirms the same, with extremely few mispredictions—chiefly between Leaf smut and nearby categories—and the rest of the predictions correct. Macro and weighted averages of 0.98 indicate the equilibrium and fairness of the model on class distributions. Such a performance is a telltale sign that the model can discriminate between visually inseparable plant diseases. Overall, this suggests a robust and consistent framework for real-world use in field diagnosis, with minimal performance gaps. Future improvements might focus on the reduction of minor confusion between closely related leaf states.

CHAPTER 6

CONCLUSION AND FUTURE ENHANCEMENT

6.1 CONCLUSION

The deep learning model developed for rice leaf disease classification works exceptionally well as a predictor with test accuracy of 98.15%. The precision, recall, F1-score, and confusion matrix performance metrics confirm that the model generalizes well across disease classes. The standout in this case is correct classification of Bacterial leaf blight and healthy and only a few misclassifications for Leaf smut. This level of accuracy is a testament to the strength of the model in real-world application, where fast and correct disease identification is critical for early intervention in agriculture. With the model performing well on all sides consistently, it proves its feasibility as a handy tool for farmers and agronomists to help identify and control diseases. The rigorous training and validation processes guaranteed that the model settled beyond its initial instability to deliver uniform results.

6.2 FEATURE ENHANCEMENT – SEVERITY PREDICTION

To further increase the utility and relevance of the system, a recommended feature enhancement is the inclusion of disease severity prediction. While the current model is capable of predicting the type of disease, the model does not consider the severity of infection, which is one of the determinants in ascertaining urgency of treatment and resource allocation. Severity prediction would involve analyzing the size, size, and texture of lesions or patterns of discoloration in leaves to classify infections as mild, moderate, or severe. This may be done with the help of additional image processing techniques or by altering the current CNN structure to yield both the disease class as well as the severity score.

REFERENCES

- [1] PrajwalGowda B.S, Nisarga M A, Rachana M,Shashank S ,Mrs. Sahana Raj B.S, "Paddy Crop Disease Detection using Machine Learning," International Journal of Engineering Research & Technology (IJERT) Special Issue - 2020 ISSN: 2278-0181 NCCDS - 2020 Conference Proceedings
- [2] M. K. Roy, P. Manna, R. Roy, P. Roy, S. Rakshit and M. K. H. Mondal, "Disease Detection in Paddy Crop using Machine Learning Techniques," 2023 International Conference on Sustainable Computing and Data Communication Systems (ICSCDS), Erode, India, 2023, pp. 287-292, doi: 10.1109/ICSCDS56580.2023.10104908.
- [3] Md. Ashiqul Islam, Md. Nymur Rahman Shuvo, Muhammad Shamsojjaman, Shazid Hasan, Md. Shahadat Hossain and Tania Khatun, "An Automated Convolutional Neural Network Based Approach for Paddy Leaf Disease Detection" International Journal of Advanced Computer Science and Applications (IJACSA), 12(1), 2021. <http://dx.doi.org/10.14569/IJACSA.2021.0120134>
- [4] Chen J, Zhang D, Nanekaran YA, Li D. "Detection of rice plant diseases based on deep transfer learning," J Sci Food Agric. 2020 May;100(7):3246-3256. doi: 10.1002/jsfa.10365. Epub 2020 Mar 14. PMID: 32124447.
- [5] Lamba S, Kukreja V, Rashid J, Gadekallu TR, Kim J, Baliyan A, Gupta D and Saini S, "A novel fine-tuned deep-learning based multi-class classifier for severity of paddy leaf diseases," Front. Plant Sci. 14:1234067, 2023, doi: 10.3389/fpls.2023.1234067
- [6] S.B. Mamun, I.J. Payel, M.T. Ahad, A.S. Atkins, B. Song, Y. Li, "Grape Guard:A YOLO-based mobile application for detecting grape leaf diseases, " Journal of Electronic Science and Technology, <https://doi.org/10.1016/j.jnlest.2025.100300>.
- [7] Arunangshu Pal, Vinay Kumar, "AgriDet: Plant Leaf Disease severity classification using agriculture detection framework," Engineering Applications of Artificial Intelligence, Volume 119, 2023, 105754, ISSN 0952-1976, doi.org/10.1016/j.engappai.2022.105754.

[8] Patel, V.K., Abhishek, K. & Selvarajan, S. "Optimized recurrent neural network-based early diagnosis of crop pest and diseases in agriculture," *Discov Computing* 27, 43 (2024). <https://doi.org/10.1007/s10791-024-09481-2>

[9] Disha Garg and Mansaf Alam, "Integration of Convolutional Neural Networks and Recurrent Neural Networks for Foliar Disease Classification in Apple Trees" *International Journal of Advanced Computer Science and Applications(ijacs)*, 13(4), 2022. <http://dx.doi.org/10.14569/IJACSA.2022.0130442>

[10] Sue Han Lee, Hervé Goëau, Pierre Bonnet and Alexis Joly. "Attention-Based Recurrent Neural Network for Plant Disease Classification," *Front. Plant Sci.*, vol 11:601250, Dec 2020, doi: 10.3389/fpls.2020.601250.

[11] S. Vidhanaarachchi, J. L. Wijekoon, W. A. S. P. Abeysiriwardhana and M. Wijesundara, "Early Diagnosis and Severity Assessment of Weligama Coconut Leaf Wilt Disease and Coconut Caterpillar Infestation Using Deep Learning-Based Image Processing Techniques," in *IEEE Access*, vol. 13, pp. 24463-24477, 2025, doi: 10.1109/ACCESS.2025.3537664.

[12] Lamba, S., Baliyan, A. & Kukreja, V. A, "novel GCL hybrid classification model for paddy diseases," *Int. j. inf. tecnol.* **15**, 1127–1136, 2023, doi.org/10.1007/s41870-022-01094-6

[13] Aboelenin, S., Elbasheer, F.A., Eltoukhy, M.M. *et al*, "A hybrid Framework for plant leaf disease detection and classification using convolutional neural networks and vision transformer," *Complex Intell. Syst.* **11**, 142 (2025). <https://doi.org/10.1007/s40747-024-01764-x>

[14] Ruchi Rani, Jayakrushna Sahoo, Sivaiah Bellamkonda, Sumit Kumar, "Attention-enhanced corn disease diagnosis using few-shot learning and VGG16," *MethodsX*, Volume 14, 2025, 103172, ISSN 2215-0161, <https://doi.org/10.1016/j.mex.2025.103172>.

- [15] Mohanty SP, Hughes DP and Salathé M, "Using Deep Learning for Image-Based Plant Disease Detection," *Front. Plant Sci.*, vol 7:1419, Sep 2016, doi: 10.3389/fpls.2016.01419
- [16] Bolanle F. Oladejo and Oladejo Olajide Ademola, "Automated Classification of Banana leaf disease using an Optimized Capsule Network Model," AIRCC Publishing Corporation, pp. 119-13, 2020, DOI: 10.5121/csit.2020.100910
- [17] M. Yogeshwari, G. Thailambal, "Automatic feature extraction and detection of plant leaf disease using GLCM features and convolutional neural networks", *Materials Today: Proceedings*, Vol. 81, Part 2, pp: 530-536, 2023. ISSN 2214-7853, <https://doi.org/10.1016/j.matpr.2021.03.700>.
- [18] Getinet Yilma, Mesfin Dagne, Mohammed Kemal Ahmed and Ravindra Babu Bellam, "Attentive Self-supervised Contrastive Learning (ASCL) for plant disease classification," *Science Direct*, Vol 25, doi.org/10.1016/j.rineng.2025.103922.
- [19] Simhadri, C. G., & Kondaveeti, H. K., "Automatic Recognition of Rice Leaf Diseases Using Transfer Learning", *Agronomy*, Vol.13, No.4, 961, 2023. <https://doi.org/10.3390/agronomy13040961>
- [20] Zamani, S.A. & Baleghi, Y., "Early/late fusion structures with optimized feature selection for weed detection using visible and thermal images of paddy fields", *Precision Agriculture*, Vol.24, pp:482–510, 2023.
- [21] W. Shafik, A. Tufail, A. Namoun, L. C. De Silva and R. A. A. H. M. Apong, "A Systematic Literature Review on Plant Disease Detection: Motivations, Classification Techniques, Datasets, Challenges, and Future Trends," in *IEEE Access*, vol. 11, pp. 59174-59203, 2023, doi: 10.1109/ACCESS.2023.3284760

- [22] A. Bhargava, A. Shukla, O. P. Goswami, M. H. Alsharif, P. Uthansakul and M. Uthansakul, "Plant Leaf Disease Detection, Classification, and Diagnosis Using Computer Vision and Artificial Intelligence: A Review", IEEE Access, vol. 12, pp. 37443-37469, Feb 2024, doi: 10.1109/ACCESS.2024.3373001.
- [23] J. Li and C. Liao, "Tea Disease Recognition Based on Image Segmentation and Data Augmentation," IEEE Access, vol. 13, pp. 19664-19677, Jan 2025, doi: 10.1109/ACCESS.2025.3534024.
- [24] Y. Dong et al., "Automatic System for Crop Pest and Disease Dynamic Monitoring and Early Forecasting," in IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing, vol. 13, pp. 4410-4418, 2020, doi: 10.1109/JSTARS.2020.3013340.

APPENDIX A1

SOURCE CODE

#Data Augmentation and Preprocessing

```
#Import libraries
```

```
import os
```

```
import cv2
```

```
import random
```

```
import numpy as np
```

```
from tqdm import tqdm
```

```
import albumentations as A
```

```
from albumentations.augmentations.transforms import RandomBrightnessContrast
```

```
from albumentations.core.composition import OneOf
```

```
# Paths
```

```
input_root = r"C:\Users\noorul\Desktop\pro\log_dataset"
```

```
output_root = r"C:\Users\noorul\Desktop\final approach\processed_dataset"
```

```
# Ensure output root exists
```

```
os.makedirs(output_root, exist_ok=True)
```

```
# Target image size and rescale factor
```

```
img_size = (224, 224)
```

```
rescale_factor = 1.0 / 255.0
```

```
# Augmentations (only for training set)
```

```
train_augment = A.Compose([
```

```
    A.Rotate(limit=10, p=0.5),
```

```
    A.ShiftScaleRotate(shift_limit=0.1, scale_limit=0, rotate_limit=0, p=0.5),
```

```
    A.HorizontalFlip(p=0.5),
```

```
    RandomBrightnessContrast(brightness_limit=0.1, contrast_limit=0.1, p=0.5)
```

```
])
```

```

# Process each subset (train, val, test)
for subset in ['train', 'val', 'test']:
    subset_input_path = os.path.join(input_root, subset)
    subset_output_path = os.path.join(output_root, subset)
    os.makedirs(subset_output_path, exist_ok=True)
    print(f"\n Processing {subset.upper()} set")

    for class_name in os.listdir(subset_input_path):
        class_input_path = os.path.join(subset_input_path, class_name)
        class_output_path = os.path.join(subset_output_path, class_name)
        os.makedirs(class_output_path, exist_ok=True)

        for img_name in tqdm(os.listdir(class_input_path), desc=f"{class_name}"):
            img_path = os.path.join(class_input_path, img_name)
            img = cv2.imread(img_path)
            img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
            img = cv2.resize(img, img_size)

            # Normalize pixel values
            img = img.astype(np.float32) * rescale_factor

            # Augment only training images
            if subset == 'train':
                augmented = train_augment(image=img)
                img = augmented['image']

            # Save image as uint8 JPG (scale back to 0–255)
            img_to_save = (img * 255).clip(0, 255).astype(np.uint8)
            save_path = os.path.join(class_output_path, img_name)
            cv2.imwrite(save_path, cv2.cvtColor(img_to_save, cv2.COLOR_RGB2BGR))

print("\n Preprocessing and augmentation complete! Check:", output_root)

```

#Dataset_Exploratory Data Analysis (EDA)

#Import libraries

import os

import cv2

import matplotlib.pyplot as plt

import seaborn as sns

import numpy as np

from collections import defaultdict

from tqdm import tqdm

Set paths

dataset_path = r"C:\Users\noorul\Desktop\another approach\dataset\train" #

change to val or test as needed

Initialize

class_counts = defaultdict(int)

image_shapes = []

corrupted_images = []

all_classes = os.listdir(dataset_path)

print(" Analyzing dataset at:", dataset_path)

Scan through dataset

for class_name in all_classes:

 class_path = os.path.join(dataset_path, class_name)

 for img_name in tqdm(os.listdir(class_path), desc=f"Checking {class_name}"):

 img_path = os.path.join(class_path, img_name)

 # Load image

 try:

 img = cv2.imread(img_path)

 if img is None:

 corrupted_images.append(img_path)

 continue


```

        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        image_shapes.append(img.shape)
        class_counts[class_name] += 1
    except:
        corrupted_images.append(img_path)

# Print results
print("\n Image count per class:")
for k, v in class_counts.items():
    print(f" {k}: {v} images")
print(f"\n Total corrupted images found: {len(corrupted_images)}")
if corrupted_images:
    print(" Sample corrupted image path:", corrupted_images[0])

# Check image shape consistency
unique_shapes = set(image_shapes)
print("\n Unique image shapes:", unique_shapes)

# Plot class distribution
plt.figure(figsize=(8, 5))
sns.barplot(x=list(class_counts.keys()), y=list(class_counts.values()), palette='Set2')
plt.title("Class Distribution in Train Set")
plt.xlabel("Class")
plt.ylabel("Number of Images")
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()

# Show random images from each class
def show_sample_images(dataset_path, class_names, num_images=3):
    plt.figure(figsize=(12, len(class_names) * 2.5))
    i = 1
    for class_name in class_names:

```

```

class_path = os.path.join(dataset_path, class_name)
images = os.listdir(class_path)
sample_imgs = np.random.choice(images, num_images, replace=False)
for img_name in sample_imgs:
    img_path = os.path.join(class_path, img_name)
    img = cv2.imread(img_path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    plt.subplot(len(class_names), num_images, i)
    plt.imshow(img)
    plt.axis("off")
    plt.title(class_name)
    i += 1
plt.tight_layout()
plt.show()

```

Show random 3 images from each class

```
show_sample_images(dataset_path, all_classes, num_images=3)
```

#Hybrid model Resnet50 and Vision Transformer Training

1. Mound drive

2. Imports

```
import os
```

```
import torch
```

```
import torchvision.transforms as transforms
```

```
from torchvision import datasets
```

```
from torchvision.models import resnet50, ResNet50_Weights
```

```
from torch.utils.data import DataLoader
```

```
import torch.nn as nn
```

```
import torch.optim as optim
```

```
from torch.optim.lr_scheduler import OneCycleLR
```

```
import matplotlib.pyplot as plt
```

```
from tqdm import tqdm
```

```
from sklearn.metrics import classification_report, confusion_matrix,
```

ConfusionMatrixDisplay

3. Parameters

```
data_path = r"C:\Users\noorul\Desktop\pro\dataset"
train_dir = os.path.join(data_path, "train")
val_dir = os.path.join(data_path, "val")
test_dir = os.path.join(data_path, "test")

batch_size = 32
num_epochs = 25
learning_rate = 1e-3
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

4. Transforms (Resize only)

```
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
])
```

5. Datasets & Loaders

```
train_dataset = datasets.ImageFolder(train_dir, transform=transform)
val_dataset = datasets.ImageFolder(val_dir, transform=transform)
test_dataset = datasets.ImageFolder(test_dir, transform=transform)

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
```

```
class_names = train_dataset.classes
num_classes = len(class_names)
```

6. Load Fusion Model

```
from torchvision.models.vision_transformer import vit_b_16, ViT_B_16_Weights
```

```

class ResNet50_ViT_Fusion(nn.Module):
    def __init__(self, num_classes):
        super(ResNet50_ViT_Fusion, self).__init__()

        # Load your fine-tuned ResNet50 model
        base_resnet = resnet50(weights=None) # No default weights because we're
loading your trained weights
        base_resnet.fc = nn.Identity() # remove the final classifier
        self.resnet = nn.Sequential(*list(base_resnet.children())[:-1]) # exclude final fc
        self.resnet_out_features = 2048

        # Load pretrained ViT
        self.vit = vit_b_16(weights=ViT_B_16_Weights.DEFAULT)
        self.vit.heads = nn.Identity() # remove classifier head
        self.vit_out_features = 768

        # Classifier Head
        self.classifier = nn.Sequential(
            nn.Linear(self.resnet_out_features + self.vit_out_features, 512),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(512, num_classes)
        )

    def forward(self, x):
        res_feat = self.resnet(x).view(x.size(0), -1) # (B, 2048)
        vit_feat = self.vit(x) # (B, 768)
        fused = torch.cat((res_feat, vit_feat), dim=1) # (B, 2816)
        return self.classifier(fused)

# Initialize model and load ResNet50 trained weights
model = ResNet50_ViT_Fusion(num_classes).to(device)

```

```
# Load your trained ResNet50 weights into the fusion model's ResNet branch
```

```
resnet_weights_path =
```

```
"/content/drive/MyDrive/paddy/new_resnet50_paddy_leaf1.pth"
```

```
resnet_state_dict = torch.load(resnet_weights_path, map_location=device)
```

```
# Load only matching keys (this avoids issues with classifier mismatch)
```

```
resnet_base = resnet50(weights=None)
```

```
resnet_base.fc = nn.Identity()
```

```
resnet_base.load_state_dict(resnet_state_dict, strict=False)
```

```
resnet_base = resnet_base.to(device) # Fix added here
```

```
model.resnet = nn.Sequential(*list(resnet_base.children())[:-1])
```

```
# 7. Loss, Optimizer, OneCycleLR Scheduler
```

```
criterion = nn.CrossEntropyLoss()
```

```
optimizer = optim.Adam(model.parameters(), lr=learning_rate)
```

```
scheduler = OneCycleLR(optimizer, max_lr=learning_rate,
```

```
steps_per_epoch=len(train_loader), epochs=num_epochs)
```

```
# 8. Metrics Tracking
```

```
train_losses = []
```

```
val_losses = []
```

```
train_accuracies = []
```

```
val_accuracies = []
```

```
learning_rates = []
```

```
# 9. Training Loop
```

```
for epoch in range(num_epochs):
```

```
    model.train()
```

```
    running_loss, correct, total = 0.0, 0, 0
```

```
    for images, labels in tqdm(train_loader, desc=f"[Epoch {epoch+1}/{num_epochs}]
```

```
Training"):
```

```
        images, labels = images.to(device), labels.to(device)
```

```

optimizer.zero_grad()
outputs = model(images)
loss = criterion(outputs, labels)
loss.backward()
optimizer.step()
scheduler.step()

running_loss += loss.item()
_, predicted = torch.max(outputs, 1)
total += labels.size(0)
correct += (predicted == labels).sum().item()

train_loss = running_loss / len(train_loader)
train_acc = 100 * correct / total
train_losses.append(train_loss)
train_accuracies.append(train_acc)
learning_rates.append(optimizer.param_groups[0]['lr'])

# Validation
model.eval()
val_loss, correct, total = 0.0, 0, 0
with torch.no_grad():
    for images, labels in val_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        loss = criterion(outputs, labels)
        val_loss += loss.item()
        _, predicted = torch.max(outputs, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

```

```

val_loss /= len(val_loader)
val_acc = 100 * correct / total
val_losses.append(val_loss)
val_accuracies.append(val_acc)

print(f"\n[Epoch {epoch+1}/{num_epochs}]")
print(f"Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.2f}%")
print(f"Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.2f}%")
print(f"Learning Rate: {optimizer.param_groups[0]['lr']:.6f}")

```

10. Plot Results

```

plt.figure(figsize=(15,5))
plt.subplot(1, 3, 1)
plt.plot(train_losses, label='Train Loss')
plt.plot(val_losses, label='Val Loss')
plt.title('Loss Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.subplot(1, 3, 2)
plt.plot(train_accuracies, label='Train Acc')
plt.plot(val_accuracies, label='Val Acc')
plt.title('Accuracy Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy (%)')
plt.legend()

plt.subplot(1, 3, 3)
plt.plot(learning_rates, label='Learning Rate')
plt.title('Learning Rate Schedule')
plt.xlabel('Epoch')
plt.ylabel('LR')

```

```
plt.legend()
plt.tight_layout()
plt.show()
```

```
# 11. Save the model
```

```
model_save_path = "/content/drive/MyDrive/paddy/Resnet50 + ViT.pth"
torch.save(model.state_dict(), model_save_path)
print(f" Model saved to {model_save_path}")
```

```
# 12. Evaluate on Test Set
```

```
model.eval()
all_preds = []
all_labels = []
```

```
with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(device)
        outputs = model(images)
        _, predicted = torch.max(outputs, 1)
        all_preds.extend(predicted.cpu().numpy())
        all_labels.extend(labels.numpy())
```

```
# Classification Report & Confusion Matrix
```

```
print("\n Classification Report:")
print(classification_report(all_labels, all_preds, target_names=class_names))
```

```
cm = confusion_matrix(all_labels, all_preds)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_names)
disp.plot(cmap="Blues", xticks_rotation=45)
plt.title(" Confusion Matrix")
plt.show()
```


#Model Evaluate on Train, Val, Test

1. Imports

```
import os
import torch
import torch.nn as nn
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
from torchvision.models import resnet50
from torchvision.models.vision_transformer import vit_b_16, ViT_B_16_Weights
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report, confusion_matrix,
ConfusionMatrixDisplay
import numpy as np
```

2. Configs

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
data_path = r"C:\Users\noorul\Desktop\pro\dataset"
batch_size = 32
```

```
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor()
])
```

3. Datasets & Loaders

```
train_dataset = datasets.ImageFolder(os.path.join(data_path, "train"),
transform=transform)
val_dataset = datasets.ImageFolder(os.path.join(data_path, "val"),
transform=transform)
test_dataset = datasets.ImageFolder(os.path.join(data_path, "test"),
transform=transform)
```

```

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=False)
val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

```

```

class_names = train_dataset.classes
num_classes = len(class_names)

```

4. Fusion Model Definition

```

class ResNet50_ViT_Fusion(nn.Module):
    def __init__(self, num_classes):
        super().__init__()

        # ResNet50 branch
        base_resnet = resnet50(weights=None)
        base_resnet.fc = nn.Identity()
        self.resnet = nn.Sequential(*list(base_resnet.children())[:-1])
        self.resnet_out_features = 2048

        # ViT branch
        self.vit = vit_b_16(weights=ViT_B_16_Weights.DEFAULT)
        self.vit.heads = nn.Identity()
        self.vit_out_features = 768

        # Classifier head
        self.classifier = nn.Sequential(
            nn.Linear(self.resnet_out_features + self.vit_out_features, 512),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(512, num_classes)
        )

    def forward(self, x):

```

```

        res_feat = self.resnet(x).view(x.size(0), -1)
        vit_feat = self.vit(x)
        fused = torch.cat((res_feat, vit_feat), dim=1)
        return self.classifier(fused)

# 5. Load Model
model = ResNet50_ViT_Fusion(num_classes).to(device)
model.load_state_dict(torch.load("/content/drive/MyDrive/paddy/fusion.pth",
map_location=device))
model.eval()

# 6. Evaluation Function
def evaluate_model(model, dataloader, dataset_name="Dataset"):
    model.eval()
    all_preds = []
    all_labels = []

    with torch.no_grad():
        for images, labels in dataloader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            _, preds = torch.max(outputs, 1)
            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())

    accuracy = 100 * np.mean(np.array(all_preds) == np.array(all_labels))
    print(f"\n {dataset_name} Accuracy: {accuracy:.2f}%")
    print(" Classification Report:")
    print(classification_report(all_labels, all_preds, target_names=class_names))

    cm = confusion_matrix(all_labels, all_preds)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=class_names)

```

```

disp.plot(cmap="Blues", xticks_rotation=45)
plt.title(f"Confusion Matrix - {dataset_name}")
plt.grid(False)
plt.show()

```

7. Evaluate on Train, Val, Test

```

evaluate_model(model, train_loader, "Train Set")
evaluate_model(model, val_loader, "Validation Set")
evaluate_model(model, test_loader, "Test Set")

```

#To Predict

```

import torch
import torch.nn as nn
from torchvision import transforms
from torchvision.models import resnet50
from torchvision.models.vision_transformer import vit_b_16, ViT_B_16_Weights
from PIL import Image
import matplotlib.pyplot as plt
from google.colab import files

```

Upload Image

```

uploaded = files.upload()

```

Class names (update if needed)

```

class_names = ['Bacterial leaf blight', 'Brown spot', 'Leaf smut', 'healthy']
num_classes = len(class_names)

```

Define Fusion Model

```

class ResNet50_ViT_Fusion(nn.Module):
    def __init__(self, num_classes):
        super().__init__()
        base_resnet = resnet50(weights=None)
        base_resnet.fc = nn.Identity()

```

```

self.resnet = nn.Sequential(*list(base_resnet.children())[:-1])
self.resnet_out_features = 2048

self.vit = vit_b_16(weights=ViT_B_16_Weights.DEFAULT)
self.vit.heads = nn.Identity()
self.vit_out_features = 768

self.classifier = nn.Sequential(
    nn.Linear(self.resnet_out_features + self.vit_out_features, 512),
    nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(512, num_classes)
)

def forward(self, x):
    res_feat = self.resnet(x).view(x.size(0), -1)
    vit_feat = self.vit(x)
    fused = torch.cat((res_feat, vit_feat), dim=1)
    return self.classifier(fused)

# Load Model
model_path = "/content/drive/MyDrive/paddy/fusion.pth"
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model = ResNet50_ViT_Fusion(num_classes).to(device)
model.load_state_dict(torch.load(model_path, map_location=device))
model.eval()

# Image transformation
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
])

```

```

# Prediction function
def predict_image(image_path):
    image = Image.open(image_path).convert("RGB")
    input_tensor = transform(image).unsqueeze(0).to(device)

    with torch.no_grad():
        outputs = model(input_tensor)
        _, predicted = torch.max(outputs, 1)
        predicted_class = class_names[predicted.item()]

# Show result
plt.imshow(image)
plt.title(f"Predicted: {predicted_class}")
plt.axis('off')
plt.show()
print(f" Predicted class: {predicted_class}")

# Predict for uploaded image(s)
for filename in uploaded.keys():
    predict_image(filename)

```

APPENDIX A2

SCREENSHOTS

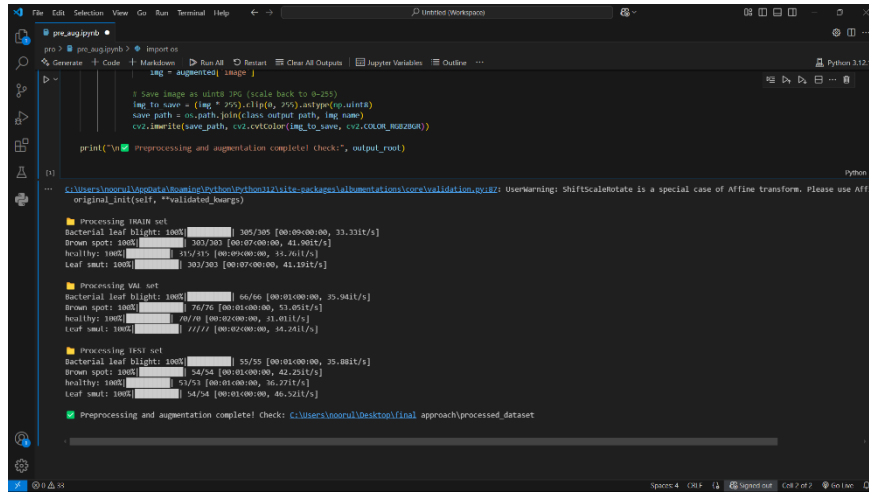


Figure A2.1: Data preprocessing Code

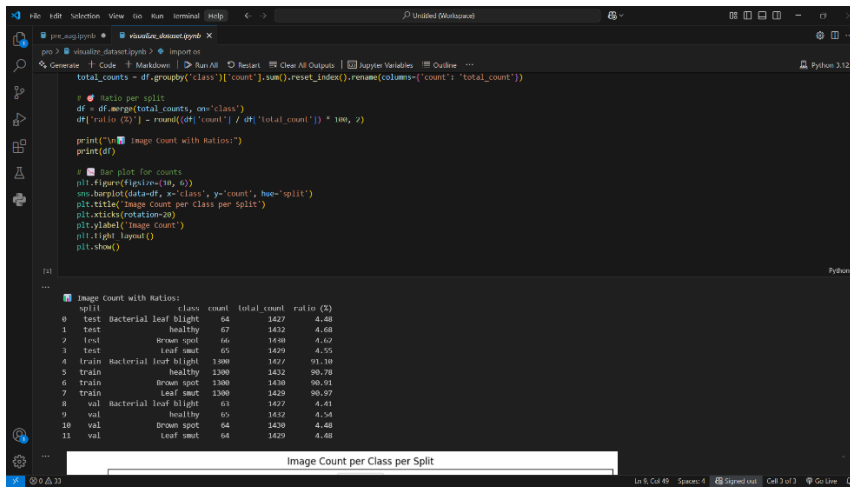


Figure A2.2: Exploratory Data Analysis (EDA) Code

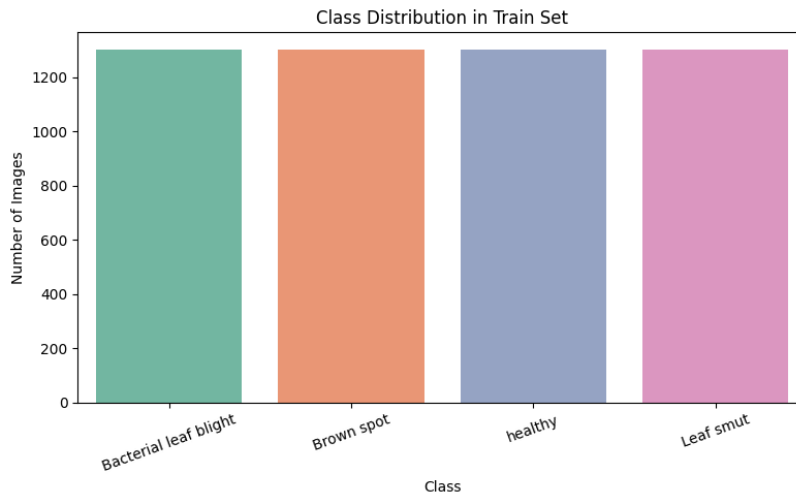


Figure A2.3: Class Distribution



Figure A2.4: Leaf Images

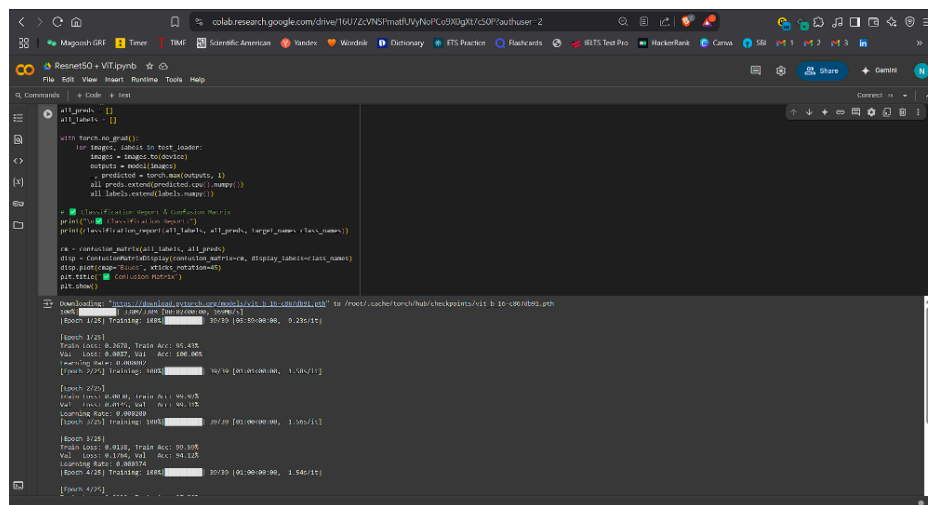


Figure A2.5: Model Training Code

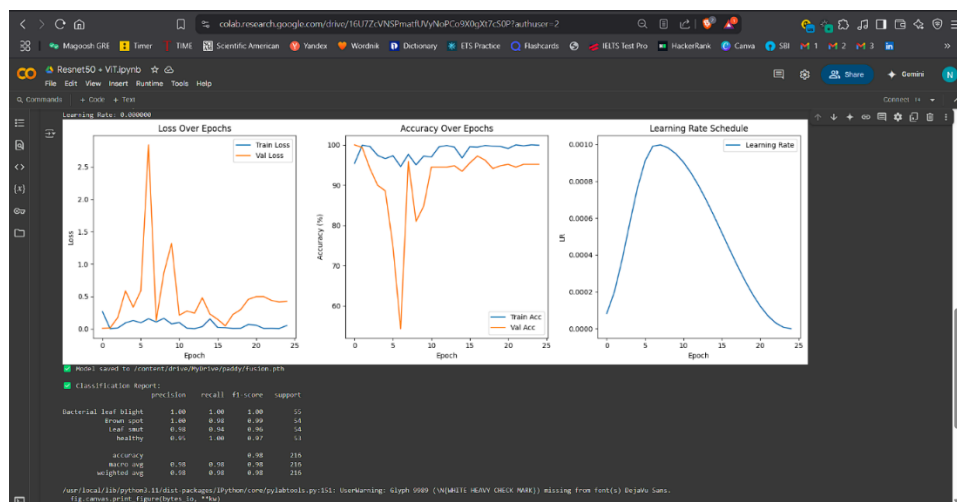


Figure A2.6: Model Evaluation

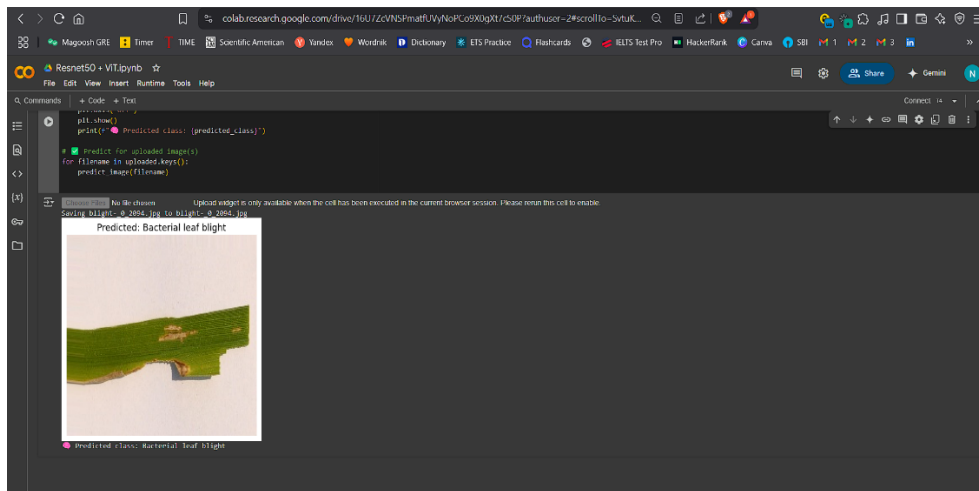


Figure A2.7: Disease Prediction Output

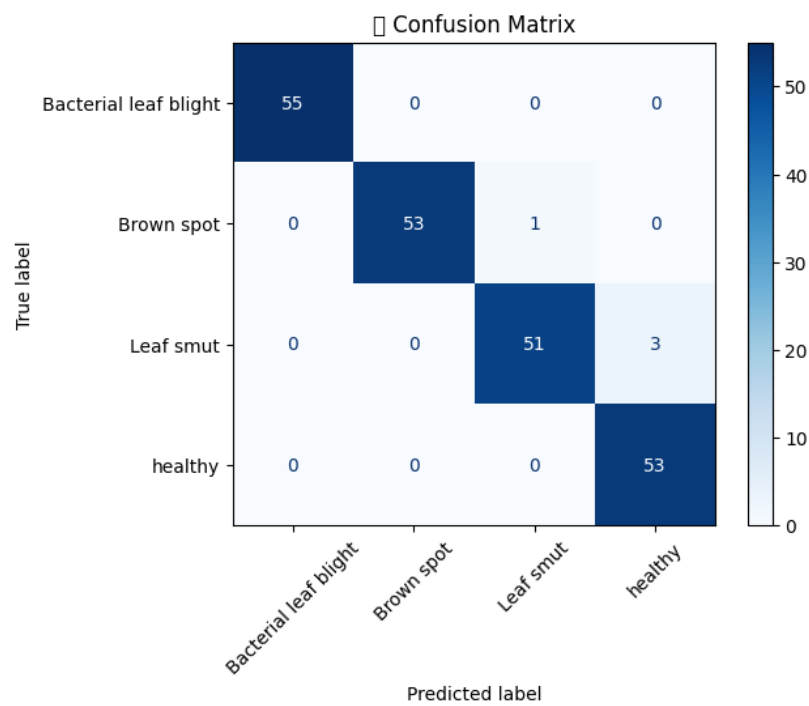


Figure A2.8: Overall Confusion Matrix

APPENDIX A3

SYSTEM REQUIREMENTS

System Requirements

To ensure successful development, training, and deployment of the rice leaf disease classification model using deep learning techniques, the following system requirements were used and are recommended for reproducibility and scalability.

1. Hardware Requirements

The computational demands of deep learning, particularly in model training and image processing tasks, require robust hardware configurations. The system used for this project is outlined below

Configuration:

- Processor: Intel Core i7 (10th Gen or above) / AMD Ryzen 7 or higher
- RAM: 16 GB or more
- Storage: 512 GB SSD or more (for storing datasets, checkpoints, and logs)
- GPU: NVIDIA RTX 2060 or higher with at least 6 GB VRAM
- Operating System: Windows 10/11 (64-bit) or Ubuntu 20.04 LTS

The training process particularly benefits from a CUDA-enabled GPU to significantly speed up the computations.

2. Software Requirements

For development and execution, a combination of programming environments and frameworks was used. The following software stack was implemented during the project:

Operating System:

- Windows 10 (64-bit) / Ubuntu 20.04 LTS (for Linux-based systems)

Development Environment:

- Python Version: Python 3.8+
- IDE/Editor: Jupyter Notebook (via Anaconda), Visual Studio Code (VSCode)

Project Management and Version Control:

- Git (for version tracking)
- GitHub (remote repository and collaboration)

Model Development Tools:

- Jupyter Notebook: For interactive code development, visualization, and debugging.
- VS Code: For structured script-based development and environment management.

3. Library and Package Requirements

Below is the list of essential Python libraries and packages used throughout the development of the classification model:

Core Libraries:

- numpy: Numerical operations and array manipulation
- pandas: Data loading and analysis
- matplotlib, seaborn: Visualization of training performance and evaluation results
- scikit-learn: Evaluation metrics like accuracy, classification report, confusion matrix

Deep Learning Framework:

- torch (PyTorch): Model building, training, and inference
- torchvision: Pretrained models, image transformations, and dataset utilities

Image Processing & Augmentation:

- PIL (Python Imaging Library): Image reading and formatting
- albumentations: Advanced image augmentation for training data

Utility Libraries:

- tqdm: Progress bars for training loop visualization
- os, glob: File system navigation
- time: Execution time measurement

TECHNICAL BIOGRAPHY



NOORUL HASSAN M U (RRN: 210081601045) is a final year undergraduate student of B.Tech in Information Technology at B.S. Abdur Rahman Crescent Institute of Science and Technology, graduating with a CGPA of 7.45. He has a strong interest in IoT, Artificial Intelligence, and smart environment automation, and is aspiring to build a career as a hands-on systems engineer. Noorul has worked on multiple hardware-based projects such as accident alert systems and deep learning-based NIDS. He also gained international exposure through an academic internship at NUS and NTU, Singapore, and contributed as a co-author to a research paper published in Springer.

Email: noorul.mu@gmail.com

Contact: +91 6374920466

plag_report

ORIGINALITY REPORT

7 %

SIMILARITY INDEX

5 %

INTERNET SOURCES

6 %

PUBLICATIONS

1 %

STUDENT PAPERS

PRIMARY SOURCES

1

www.mdpi.com

Internet Source

1 %

2

jisem-journal.com

Internet Source

1 %

3

"Proceedings of 3rd International Conference on Smart Computing and Cyber Security", Springer Science and Business Media LLC, 2024

Publication

1 %

4

Mehdi Ghayoumi. "Generative Adversarial Networks in Practice", CRC Press, 2023

Publication

1 %

5

"Computer Vision and Machine Learning in Agriculture, Volume 2", Springer Science and Business Media LLC, 2022

Publication

<1 %

6

Muhammad Ayub Sabir, Fatima Ashraf. "Development of a novel deep convolutional neural network model for early detection of brain stroke using ct scan images", Multimedia Tools and Applications, 2024

Publication

<1 %

7

isteonline.org

Internet Source

<1 %

8	"Image and Graphics Technologies and Applications", Springer Science and Business Media LLC, 2018 Publication	<1 %
9	Submitted to VIT University Student Paper	<1 %
10	assets-eu.researchsquare.com Internet Source	<1 %
11	"Proceedings of the 9th International Conference on Computational Science and Technology", Springer Science and Business Media LLC, 2023 Publication	<1 %
12	oar.icrisat.org Internet Source	<1 %
13	issuu.com Internet Source	<1 %
14	pubs.aip.org Internet Source	<1 %
15	www.jetir.org Internet Source	<1 %
16	Submitted to London Metropolitan University Student Paper	<1 %
17	ijarsct.co.in Internet Source	<1 %
18	faculty.daffodilvarsity.edu.bd Internet Source	<1 %
19	reason.town Internet Source	<1 %

<1 %

20

www.researchsquare.com
Internet Source

<1 %

21

"Recent Advances on Soft Computing and Data Mining", Springer Science and Business Media LLC, 2024
Publication

<1 %

22

addepto.com
Internet Source

<1 %

23

pescemandya.org
Internet Source

<1 %

24

Submitted to Heriot-Watt University
Student Paper

<1 %

25

S Kavın Kumar, T Kowshik, M Krishna Harini, R. Kingsy Grace. "Disease Classification in Paddy Crop Leaves Using Deep Learning", 2023 7th International Conference on Electronics, Communication and Aerospace Technology (ICECA), 2023
Publication

<1 %

26

mmcalumni.ca
Internet Source

<1 %

Exclude quotes On

Exclude matches < 14 words

Exclude bibliography On

NOORAL HASSAN

plag_report.pdf

 My Files My Files University

Document Details

Submission ID**trn:oid::17268:92645772****Submission Date****Apr 24, 2025, 10:11 AM GMT+5:30****Download Date****Apr 24, 2025, 10:12 AM GMT+5:30****File Name****plag_report.pdf****File Size****4.3 MB****38 Pages****9,004 Words****52,036 Characters**

16% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **149** Not Cited or Quoted 15%
Matches with neither in-text citation nor quotation marks
-  **0** Missing Quotations 0%
Matches that are still very similar to source material
-  **5** Missing Citation 1%
Matches that have quotation marks, but no in-text citation
-  **3** Cited and Quoted 1%
Matches with in-text citation present, but no quotation marks

Top Sources

- 9%  Internet sources
- 8%  Publications
- 13%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 149** Not Cited or Quoted 15%
Matches with neither in-text citation nor quotation marks
- 0** Missing Quotations 0%
Matches that are still very similar to source material
- 5** Missing Citation 1%
Matches that have quotation marks, but no in-text citation
- 3** Cited and Quoted 1%
Matches with in-text citation present, but no quotation marks

Top Sources

- 9% Internet sources
- 8% Publications
- 13% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	www.mdpi.com	1%
2	Submitted works	University of Wales, Bangor on 2024-05-30	<1%
3	Internet	jisem-journal.com	<1%
4	Publication	"Proceedings of 3rd International Conference on Smart Computing and Cyber Sec...	<1%
5	Submitted works	Letterkenny Institute of Technology on 2024-08-30	<1%
6	Internet	arxiv.org	<1%
7	Internet	assets-eu.researchsquare.com	<1%
8	Submitted works	mctrariv on 2025-04-23	<1%
9	Internet	dspace.daffodilvarsity.edu.bd:8080	<1%
10	Internet	www.semanticscholar.org	<1%

11	Submitted works	Mepco Schlenk Engineering college on 2025-04-23	<1%
12	Internet	isteonline.org	<1%
13	Submitted works	University Tun Hussein Onn Malaysia on 2025-01-20	<1%
14	Submitted works	Anna University on 2025-01-08	<1%
15	Submitted works	VIT University on 2025-04-03	<1%
16	Internet	eprints.leedsbeckett.ac.uk	<1%
17	Publication	"Proceedings of the 9th International Conference on Computational Science and ...	<1%
18	Internet	mmcalumni.ca	<1%
19	Internet	www.altmetric.com	<1%
20	Submitted works	The University of the West of Scotland on 2025-04-21	<1%
21	Publication	Lakshmi Narayana Thalluri, Sai Divya Adapa, Priyanka D, Sri Nithya N et al. "Dron...	<1%
22	Submitted works	VIT University on 2025-03-25	<1%
23	Internet	oar.icrisat.org	<1%
24	Submitted works	Gisma University of Applied Sciences GmbH on 2024-12-22	<1%

25	Internet	issuu.com	<1%
26	Submitted works	Swinburne University of Technology on 2023-11-06	<1%
27	Internet	pubs.aip.org	<1%
28	Submitted works	Brunel University on 2024-09-20	<1%
29	Submitted works	London Metropolitan University on 2024-05-07	<1%
30	Internet	ijarsct.co.in	<1%
31	Internet	pubmed.ncbi.nlm.nih.gov	<1%
32	Internet	reason.town	<1%
33	Internet	www.frontiersin.org	<1%
34	Internet	www.researchsquare.com	<1%
35	Publication	"Recent Advances on Soft Computing and Data Mining", Springer Science and Bus...	<1%
36	Submitted works	Liverpool John Moores University on 2022-06-06	<1%
37	Submitted works	The Robert Gordon University on 2020-05-17	<1%
38	Internet	addepto.com	<1%

39	Submitted works	Heriot-Watt University on 2024-11-21	<1%
40	Publication	S Kavin Kumar, T Kowshik, M Krishna Harini, R. Kingsy Grace. "Disease Classificati...	<1%
41	Internet	ouci.dntb.gov.ua	<1%
42	Submitted works	Coventry University on 2025-04-10	<1%
43	Submitted works	University of Nottingham on 2024-12-17	<1%
44	Publication	Xiaoli Zhao, Minping Jia. "A New Local-Global Deep Neural Network and its Applic...	<1%
45	Internet	diversedaily.com	<1%
46	Internet	ijirt.org	<1%
47	Submitted works	Heriot-Watt University on 2024-04-16	<1%
48	Submitted works	Hong Kong University of Science and Technology on 2025-04-18	<1%
49	Submitted works	University of Westminster on 2025-04-16	<1%
50	Internet	repository.uantwerpen.be	<1%
51	Internet	www.ijraset.com	<1%
52	Internet	arccjournals.com	<1%

53	Internet	baadalsg.inflibnet.ac.in	<1%
54	Internet	export.arxiv.org	<1%
55	Internet	ibl.kb.nl	<1%
56	Internet	researchoutput.csu.edu.au	<1%
57	Internet	www.biorxiv.org	<1%
58	Internet	www.ijisae.org	<1%
59	Publication	Amit Bijlwan, Rajeev Ranjan, Shweta Pokhariyal, Ajit Govind et al. "Enhancing Ric...	<1%
60	Submitted works	Columbia University on 2024-12-27	<1%
61	Submitted works	Glasgow Caledonian University on 2025-01-03	<1%
62	Publication	Nusrat Islam, Wenjiang Huang, Houlong L. Zhuang. "Machine learning for phase ...	<1%
63	Publication	Unber Zahra, Muhammad Attique Khan, Majed Alhaisoni, Areej Alasiry, Mehrez M...	<1%
64	Submitted works	University of East London on 2023-05-10	<1%
65	Submitted works	University of Sheffield on 2024-12-19	<1%
66	Internet	www.benthamdirect.com	<1%

67	Internet	www.ijert.org	<1%
68	Publication	D. Lakshmi, Ravi Shekhar Tiwari, Rajesh Kumar Dhanaraj, Seifedine Kadry. "Explai...	<1%
69	Submitted works	Daffodil International University on 2024-07-10	<1%
70	Submitted works	Higher Education Commission Pakistan on 2015-05-22	<1%
71	Submitted works	Higher Education Commission Pakistan on 2022-02-27	<1%
72	Publication	Muhammad Ayub Sabir, Fatima Ashraf. "Development of a novel deep convolutio...	<1%
73	Publication	Shubham Nain, Neha Mittal, Gajendra Singh. "Chapter 20 Efficient Rice Disease Cl...	<1%
74	Submitted works	Universiti Tunku Abdul Rahman on 2023-09-10	<1%
75	Submitted works	University of East London on 2020-09-09	<1%
76	Submitted works	University of Essex on 2024-09-18	<1%
77	Submitted works	University of Greenwich on 2024-09-06	<1%
78	Submitted works	University of Limerick on 2024-03-19	<1%
79	Submitted works	University of Nottingham on 2025-04-18	<1%
80	Submitted works	University of Western Ontario - Main on 2025-04-21	<1%

81	Submitted works	Visvesvaraya Technological University on 2014-11-19	<1%
82	Internet	Inu.diva-portal.org	<1%
83	Internet	thesai.org	<1%
84	Internet	www.gaston-mazzacane.com	<1%
85	Submitted works	Addis Ababa University on 2024-08-11	<1%
86	Submitted works	Bangladesh University of Professionals on 2024-03-12	<1%
87	Submitted works	Erasmus University of Rotterdam on 2019-04-26	<1%
88	Submitted works	Anna University on 2024-11-16	<1%
89	Submitted works	Khulna University on 2023-11-16	<1%
90	Submitted works	University of Bedfordshire on 2024-07-11	<1%
91	Publication	Wei Song, Ling Xin, Jiemei Wang. "A grading method for Kayser Fleischer ring ima...	<1%